



Volatility3 Malware Analysis Manual 2026

**Userland, Network, Kernel Rootkits & Professional Memory
Forensics**

Expanded Edition: Ransomware, AWS, Azure & Hybrid Incident Response

0xAllow

01.08.2026

Defensive Use, Scope and Version Notice

This manual is written for authorized incident response, digital forensics, malware analysis, threat hunting, security engineering, education and controlled laboratory research. It does not authorize access to systems, memory, credentials, cloud tenants or data outside the reader's lawful scope. Memory images frequently contain authentication material, private keys, regulated data, personal communications and proprietary information. Treat every dump as sensitive evidence.

The command baseline is **Volatility 3 version 2.28.0 stable**, which the official release channel identifies as the latest stable release available at the manual's publication date. The authoritative local reference remains the analyst's installed binary: `vol.exe -version`, `vol.exe -h`, and `vol.exe <PLUGIN> -h`. Plugin availability and option names can change between releases, documentation builds and third-party distributions. This manual uses full class-qualified plugin identifiers where they are exposed by version 2.28.0.[\[1, 2\]](#)

This edition integrates memory forensics with endpoint, network, ransomware, AWS, Azure and hybrid incident response. It distinguishes facts observed in evidence from hypotheses, inferences and external intelligence. A clean memory image cannot prove the universal absence of compromise. Firmware, peripheral devices, inactive malware, encrypted pages, unacquired physical ranges, provider-controlled cloud layers and acquisition side effects remain outside or at the edge of what one dump can establish.

CAUTION

Do not upload complete memory images to public scanning services. Do not run destructive containment commands before preserving volatile evidence unless human safety, critical operations or an active destructive event requires immediate action. Document every action, timestamp and operator decision.

Contents

Defensive Use, Scope and Version Notice	i
1 Executive Overview	1
1.1 Purpose	1
1.2 The central rule: correlation beats isolated anomalies	1
1.3 Triage versus deep analysis	2
1.4 Confidence language	2
1.5 2026 operating context	2
2 Legal, Safety and Evidence Requirements	3
2.1 Authorization and scope	3
2.2 Safety hierarchy	3
2.3 Chain of custody	3
2.4 Acquisition contamination	4
2.5 Credential and privacy handling	4
2.6 Malware-lab controls	4
3 Professional Lab Architecture	5
3.1 Reference architecture	5
3.2 Storage planning	5
3.3 Evidence directory standard	5
3.4 Windows analysis host	6
3.5 Linux analysis host	6
3.6 macOS analyst host	6
3.7 Virtual-machine controls	6
3.8 Isolated networking	6
4 Free and Verified Memory Acquisition	8
4.1 Acquisition strategy	8
4.2 Windows: WinPmem	8
4.3 Windows: complete crash dump	8
4.4 Linux: AVML	9

4.5	Linux: LiME	9
4.6	Linux: kdump and vmcore	9
4.7	Virtual machines	9
4.8	Hardware DMA	10
4.9	macOS acquisition reality	10
4.10	Cloud workloads	10
5	Volatility3 Installation and Environment	11
5.1	Version pinning	11
5.1.1	Windows PowerShell	11
5.1.2	Linux and macOS host	11
5.2	Source installation	11
5.3	Offline installation	12
5.4	Windows symbols	12
5.5	Linux and macOS symbols	12
5.6	Plugin namespace and CLI errors	12
5.7	Output renderers and logging	12
5.8	Third-party plugins	13
6	Memory Dump Preparation and Validation	14
6.1	Preserve the original	14
6.1.1	PowerShell	14
6.1.2	Bash	14
6.2	Identify the format	14
6.3	Initial validation	14
6.4	Recognizing corruption and smearing	15
6.5	Case manifest	15
7	Professional Analysis Methodology	16
7.1	The evidence funnel	16
7.2	Phase 0: define hypotheses	16
7.3	Phase 1: establish system identity	17
7.4	Phase 2: process inventory	17
7.5	Phase 3: ancestry and execution context	17
7.6	Phase 4: network mapping	17
7.7	Phase 5: userland depth	17
7.8	Phase 6: persistence	17
7.9	Phase 7: kernel	18
7.10	Phase 8: timeline and correlation	18
7.11	Phase 9: reporting	18

7.12	Escalation matrix	18
8	Windows Runtime Structures for Analysts	19
8.1	Why structure knowledge matters	19
8.2	Processes and threads	19
8.3	PEB, loader lists and VADs	19
8.4	Handles and objects	19
8.5	Kernel modules, driver objects and callbacks	20
8.6	Security context	20
8.7	Protected processes and VBS	20
9	Userland Malware Analysis	21
9.1	Process inventory and cross-view analysis	21
9.1.1	How to read the output	21
9.1.2	Suspicious indicators	21
9.1.3	Common mistakes	21
9.2	Hierarchy and command-line analysis	22
9.3	Security identifiers, privileges and sessions	22
9.4	DLL and module analysis	22
9.5	VAD analysis	22
9.6	Injected-code detection with malfind	23
9.7	Thread analysis	23
9.8	Hollowing, ghosting and PEB masquerading	24
9.9	Direct, indirect and unhooked system calls	24
9.10	ETW patching and import analysis	24
9.11	Handles, mutants and file objects	24
9.12	Environment variables	24
9.13	YARA scanning	25
9.14	Extraction and reverse-engineering handoff	25
9.15	Credential artifact awareness	25
10	Network Artifact Analysis	26
10.1	Objectives and limitations	26
10.2	Windows commands	26
10.3	Interpretation order	26
10.4	Listening ports	27
10.5	C2 indicators	27
10.6	DNS limitations	27
10.7	Linux network analysis	27
10.8	macOS network analysis	27

10.9 Professional example	28
11 Ransomware Investigation and Response	29
11.1 Why ransomware requires a separate workflow	29
11.2 The first-hour ransomware priorities	29
11.3 Ransomware evidence model	30
11.4 Memory-forensics objectives	30
11.4.1 Identify the encryptor and launch chain	30
11.4.2 Find unpacked code and configuration	30
11.4.3 Encryption-key awareness	30
11.4.4 Lateral movement	31
11.4.5 Backup and recovery disruption	31
11.5 Endpoint-forensics objectives	31
11.6 Network-forensics objectives	31
11.7 Cloud ransomware	32
11.8 Containment decision points	32
11.9 Real incident lessons	32
11.9.1 Colonial Pipeline	32
11.9.2 Change Healthcare	32
11.9.3 ESXiArgs	33
11.10 Ransomware reporting	33
12 Ransomware Deep Technical Playbooks	34
12.1 Investigate the operation, not only the encryptor	34
12.2 Role-based incident organization	34
12.3 The representative-system acquisition plan	35
12.4 Windows encryptor triage	35
12.4.1 Process and file-impact correlation	35
12.4.2 Command-line indicators for recovery inhibition	36
12.4.3 Ransom-note and extension correlation	36
12.4.4 Cryptographic material	36
12.5 Linux ransomware playbook	37
12.5.1 First pass	37
12.5.2 Systemd and scheduled execution	37
12.5.3 Databases and application consistency	37
12.6 VMware ESXi and virtualization-platform ransomware	37
12.7 Double-extortion investigation	38
12.7.1 Volume estimation	38
12.8 Identity-first ransomware scoping	38
12.9 Active Directory evidence plan	38

12.10	AWS ransomware event families	39
12.11	Azure ransomware event families	39
12.12	Backup compromise assessment	39
12.13	Clean-room recovery	40
12.14	Ransomware evidence matrix	40
12.15	Ransomware severity and confidence	42
12.16	Ransom-payment evidence	42
13	Network Forensics Beyond the Memory Image	43
13.1	Evidence layers	43
13.2	Connection state interpretation	44
13.3	NAT and address translation	44
13.4	DNS and encrypted DNS	44
13.5	TLS analysis	44
13.6	QUIC and HTTP/3	45
13.7	Packet-capture triage with TShark	45
13.8	Zeek workflow	45
13.9	Beaconing analysis	45
13.10	Lateral-movement patterns	46
13.11	Exfiltration analysis	46
13.12	Cloud network evidence	46
13.12.1	AWS	46
13.12.2	Azure	46
13.13	Network finding template	46
14	Reverse Engineering Memory-Derived Artifacts	47
14.1	Why memory-derived files are different	47
14.2	Extraction provenance record	47
14.3	Initial identification	47
14.4	PE analysis	48
14.5	ELF analysis	48
14.6	Mach-O analysis	48
14.7	Thread-centric disassembly	48
14.8	Configuration recovery	49
14.9	FLOSS, capa and Ghidra	49
14.10	Memory-to-disk comparison	49
14.11	Direct-syscall interpretation	49
14.12	YARA engineering from reverse engineering	50
14.13	Safe dynamic analysis	50
14.14	Reverse-engineering finding example	50

15 Kernel and Rootkit Analysis	51
15.1 Defensive objective	51
15.2 Module and driver inventory	51
15.3 IRP dispatch analysis	51
15.4 Kernel callbacks	51
15.5 SSDT	52
15.6 Orphan kernel threads	52
15.7 Timers and unloaded modules	52
15.8 Device tree and symbolic links	52
15.9 Big pools and targeted pool scanning	52
15.10 Driver extraction and live verification	53
15.11 Rootkit decision rule	53
15.12 Linux rootkit checks	53
15.13 macOS kernel checks	53
16 Windows Memory Forensics: Complete Investigation Runbook	54
16.1 Scope and operating assumptions	54
16.2 Stage 0: case directory and immutable baseline	54
16.3 Stage 1: validate the image	55
16.3.1 Validation questions	55
16.4 Stage 2: process census and cross-view comparison	55
16.4.1 Interpretation sequence	55
16.5 Stage 3: threads, handles and runtime context	55
16.6 Stage 4: module and image integrity	56
16.7 Stage 5: VAD and injection analysis	56
16.7.1 VAD triage matrix	56
16.8 Stage 6: persistence and execution history	57
16.9 Stage 7: file objects and targeted extraction	57
16.10 Stage 8: credential artifact awareness	57
16.11 Stage 9: closeout criteria	58
17 Linux Memory Forensics	59
17.1 Why exact symbols matter	59
17.2 Symbol workflow	59
17.3 Process triage	59
17.4 Process memory and injected code	60
17.5 Files, mounts and page cache	60
17.6 Network analysis	60
17.7 Shell and interactive-session artifacts	60
17.8 Kernel and rootkit workflow	61

17.9 Container and Kubernetes considerations	61
17.10Linux acquisition limitations	61
18 macOS Memory Forensics: Realistic 2026 Scope	62
18.1 Support boundary	62
18.2 Acquisition reality	62
18.3 Symbol and plugin workflow	62
18.4 Kernel-oriented checks	63
18.5 Professional alternatives	63
19 Endpoint Forensics Integration	64
19.1 Memory is not a standalone investigation	64
19.2 Windows endpoint evidence	64
19.3 Linux endpoint evidence	64
19.4 macOS endpoint evidence	65
19.5 Static and reverse-engineering workflow	65
19.6 Signer and hash verification	65
20 AWS Security Incident Response and Forensics	66
20.1 Cloud evidence model	66
20.2 Preparation architecture	66
20.3 Immediate containment without destroying evidence	67
20.4 EBS snapshot acquisition	67
20.5 CloudTrail preservation and querying	67
20.6 Identity-chain analysis	67
20.7 Guest memory and EC2 triage	68
20.8 AWS ransomware investigation	68
20.9 AWS evidence checklist	68
21 Azure Security Incident Response and Forensics	70
21.1 Azure evidence model	70
21.2 Isolation through NSG reassignment	70
21.3 Managed-disk snapshot acquisition	70
21.4 Activity Log collection	71
21.5 Entra identity investigation	71
21.6 KQL examples for defensive hunting	71
21.7 Guest memory and VM triage	72
21.8 Azure ransomware investigation	72
21.9 Microsoft Sentinel investigation workflow	72
22 On-Premises, Cloud and Hybrid Investigations	73

22.1	Core differences	73
22.2	The hybrid evidence graph	73
22.3	Containment order in hybrid ransomware	74
23	Timeline Construction and Evidence Correlation	75
23.1	Time fields are not equivalent	75
23.2	Volatility timeliner	75
23.3	Correlation keys	75
23.4	Memory-smearing controls	76
23.5	Hypothesis table	76
24	IOC Extraction, Threat Hunting and Detection Engineering	77
24.1	IOC hierarchy	77
24.2	Extraction workflow	77
24.3	YARA from memory findings	77
24.4	Sigma and log mapping	78
24.5	MITRE ATT&CK mapping	78
24.6	Enterprise hunting package	78
25	Verified Incident Lessons	80
25.1	Use of public incidents	80
25.2	Colonial Pipeline ransomware response	80
25.2.1	Defensive lessons	80
25.2.2	Comparable investigation plan	80
25.3	Change Healthcare disruption	80
25.3.1	Defensive lessons	81
25.4	ESXiArgs campaign	81
25.4.1	Defensive lessons	81
25.5	MGM Resorts and identity-centric intrusion	81
25.5.1	Defensive lessons	81
25.6	Capital One cloud breach	81
25.6.1	Defensive lessons	81
25.7	Midnight Blizzard and OAuth application abuse	82
25.7.1	Defensive lessons	82
25.8	Storm-0558 and cloud-token assurance	82
25.8.1	Defensive lessons	82
26	Lab-Safe Case Studies	83
26.1	Case A: userland ransomware precursor	83
26.1.1	Scenario	83
26.1.2	Questions	83

26.1.3 Workflow	83
26.1.4 Synthetic output	83
26.1.5 Short finding	84
26.2 Case B: suspicious cloud-connected process	84
26.2.1 Scenario	84
26.2.2 Workflow	84
26.2.3 Conclusion	84
26.3 Case C: suspected Windows kernel rootkit	84
26.3.1 Scenario	84
26.3.2 Workflow	84
26.4 Case D: Azure identity and ransomware hybrid	85
26.4.1 Scenario	85
26.4.2 Method	85
27 Professional Reporting, Remediation and Peer Review	86
27.1 Report architecture	86
27.2 Finding template	86
27.3 Executive writing rules	87
27.4 Remediation principles	87
27.5 Peer review	87
28 Troubleshooting and Failure Analysis	88
28.1 Plugin identifier errors	88
28.2 Missing Windows symbols	88
28.3 Linux symbol mismatch	88
28.4 Empty output	88
28.5 Corrupt or smeared dump	89
28.6 VM snapshot issues	89
28.7 CSV and shell redirection	89
28.8 Operational permission failures	89
29 Professional Workflow Checklists	90
29.1 Pre-incident readiness	90
29.2 Acquisition checklist	90
29.3 Triage checklist	90
29.4 Deep-analysis checklist	91
29.5 Reporting checklist	91
30 Command Reference	92
30.1 Environment and output	92
30.2 Windows triage	92

30.3 Windows modules and injection	93
30.4 Windows network and persistence	93
30.5 Windows kernel	94
30.6 Linux reference	94
30.7 macOS reference	95
31 Final Recommendations for 2026	96
31.1 Build competence in layers	96
31.2 Maintain a versioned laboratory	96
31.3 Practice evidence-driven restraint	96
31.4 Recommended core references	96
31.5 Closing principle	97
A Evidence Manifest Template	98
B Chain-of-Custody Template	99
C Finding Confidence Matrix	100
D Glossary	101
E Version and Validation Record	102
Bibliography	103

List of Tables

Executive Overview

1.1 Purpose

This manual provides a professional, repeatable method for investigating volatile memory across Windows, Linux and macOS, while connecting memory evidence to endpoint telemetry, network artifacts, ransomware response and cloud control-plane evidence. It is designed for incident responders, malware analysts, digital forensic examiners, SOC analysts, threat hunters, reverse engineers and security engineers who need conclusions that survive technical review.

Memory forensics answers questions that disk-only analysis often cannot answer reliably:

- Which processes and threads existed at acquisition time?
- What command lines, loaded modules, tokens, handles and network endpoints were associated with those processes?
- Was executable code present in private or anonymous memory?
- Were processes hidden, hollowed, ghosted or masquerading?
- Were kernel drivers, callbacks, IRP handlers, timers or system-call tables anomalous?
- Which credentials, encryption material, decrypted configuration or runtime strings may have been present?
- Which observations correlate with endpoint, network, identity and cloud logs?

1.2 The central rule: correlation beats isolated anomalies

No single plugin output should be treated as a verdict. A private executable region can be generated by a browser, .NET, Java, an EDR sensor, an anti-cheat driver or malware. A driver found by pool scanning but absent from the active module list can be a recently unloaded driver, stale pool memory, a scanning false positive or a rootkit. A terminated process recovered by scanning is not automatically hidden.

High-confidence findings are built from multiple independent observations. For example:

Private executable VAD + thread start inside the VAD + process/module mismatch + suspicious parentage + external connection + matching YARA hit = strong injection finding.

1.3 Triage versus deep analysis

Triage identifies the small set of processes, modules, endpoints and persistence artifacts that deserve attention. It should be fast, reproducible and broad. **Deep analysis** validates those candidates through VAD boundaries, thread start addresses, loader-list comparison, extraction, static analysis, reverse engineering, timeline reconstruction and external telemetry.

A professional workflow does not dump every process or run every plugin without a hypothesis. It creates a reproducible evidence funnel: broad inventory, anomaly selection, targeted validation, extraction, correlation and reporting.

1.4 Confidence language

Use controlled language throughout the investigation:

Term	Meaning
Confirmed	Direct evidence establishes the behavior or artifact, and reasonable alternative explanations have been excluded.
Probable	Multiple independent indicators support the conclusion, but one or more material uncertainties remain.
Suspicious	The artifact is unusual and warrants further analysis; it is not sufficient for attribution or compromise confirmation.
Inconclusive	Available evidence cannot distinguish between competing explanations.
Benign/expected	The artifact is consistent with a known product, operating-system mechanism or validated baseline.

1.5 2026 operating context

Incident response in 2026 is no longer limited to a single endpoint. Ransomware investigations commonly span identity providers, virtualization platforms, backup infrastructure, cloud control planes, SaaS services and third-party dependencies. NIST SP 800-61 Revision 3 places incident response within continuous cybersecurity risk management rather than treating it as an isolated technical phase. NIST's 2026 ransomware profile further emphasizes that ransomware response includes both technical and business actions, including legal, regulatory, contractual and communication requirements.[3, 4]

The practical consequence is direct: memory evidence is one evidence plane. It must be correlated with endpoint logs, EDR telemetry, network data, cloud audit logs, identity records, snapshots, backups and business-impact information.

DECISION RULE

A memory finding is reportable when it changes scope, containment, eradication, recovery, attribution confidence or legal/regulatory decision-making. Interesting but operationally irrelevant anomalies belong in analyst notes, not the executive conclusion.

Legal, Safety and Evidence Requirements

2.1 Authorization and scope

Obtain written authorization that identifies systems, accounts, cloud subscriptions, time ranges, allowed acquisition methods, handling restrictions and escalation contacts. Cloud investigations must also define which accounts, regions, subscriptions, tenants and service-provider support channels are in scope.

NIST SP 800-86 recommends integrating forensic techniques into incident response under management and legal guidance because jurisdiction, privacy, labor, contractual and regulatory obligations vary.[5]

2.2 Safety hierarchy

When an incident is active, decisions must balance evidence preservation against operational harm. Use this hierarchy:

1. Protect human safety and critical physical processes.
2. Stop ongoing destructive or exfiltration activity when delay would materially increase harm.
3. Preserve volatile evidence before shutdown or reboot when operationally safe.
4. Preserve cloud control-plane and identity logs before retention windows expire.
5. Create disk snapshots and immutable evidence copies.
6. Contain with the least destructive control that is effective.

2.3 Chain of custody

At minimum, record:

- case identifier and evidence identifier;
- source hostname, serial number, cloud resource ID, region and owner;
- acquisition tool, version, executable hash and command line;
- operator, start time, completion time and timezone;
- destination media and filesystem;
- source and destination hashes;

- transfers, copies, access events and analysis actions;
- known deviations, errors and limitations.

2.4 Acquisition contamination

Every live acquisition changes memory. A driver is loaded, a process is created, files are opened, pages are touched and timestamps advance. Acquisition may also create temporary driver names, service objects or network traffic. Record these artifacts before analysis so they are not mistaken for malware.

FIELD NOTE

A live dump is not an atomic photograph. Pages are captured over time while the system continues to execute. Process creation times, exit times, linked lists and pool structures can therefore represent slightly different moments. This is memory smearing. Corrupted PIDs, impossible timestamps or inconsistent cross-view results must be evaluated against acquisition duration before they are called rootkit evidence.

2.5 Credential and privacy handling

Memory can contain passwords, hashes, tickets, browser sessions, API keys, cloud tokens, encryption keys and private data. Credential-related plugins and extracted strings must be used only under explicit authorization. Store results in restricted evidence locations, minimize distribution, and document who accessed them. Do not include raw secrets in routine reports; report the type, location, exposure and remediation requirement.

2.6 Malware-lab controls

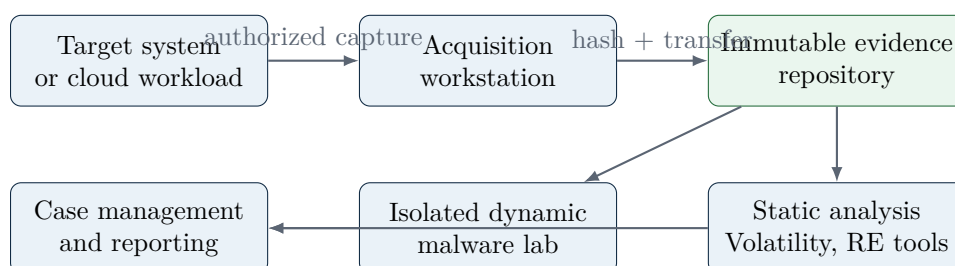
A safe lab should include:

- dedicated analysis hosts or disposable VMs;
- host-only or fully isolated networks;
- disabled shared clipboard, drag-and-drop and shared folders during execution;
- no production credentials or personal accounts;
- separate evidence storage from malware execution storage;
- snapshots taken before execution and reverted after evidence export;
- controlled DNS/HTTP simulation when dynamic analysis is authorized;
- outbound firewall controls and packet capture;
- synchronized UTC clocks without direct production-domain trust.

Professional Lab Architecture

3.1 Reference architecture

A practical DFIR lab separates acquisition, evidence storage, static analysis, dynamic analysis and reporting.



3.2 Storage planning

A single 64 GB RAM acquisition can require substantially more than 64 GB of working storage when the analyst keeps the original, a working copy, extracted processes, VAD dumps, CSV output, strings, YARA results and rendered reports. A practical planning ratio is three to five times installed RAM per acquisition, plus disk snapshots and logs.

Use filesystems that support large files, strong access control and integrity monitoring. Avoid FAT32. For long-term evidence, use redundant storage, encryption at rest and immutable retention where required.

3.3 Evidence directory standard

```

CASE-2026-0042/
  00_admin/
    authorization.pdf
    chain_of_custody.csv
    analyst_notes.md
  01_original/
    host01_20260801T141200Z.raw
    host01_20260801T141200Z.sha256
  02_working/
    host01_working.raw
  03_symbols/
  04_volatility/
    triage/
    userland/
  
```

```
network/  
kernel/  
persistence/  
timeline/  
05_extracted/  
06_endpoint/  
07_network/  
08_cloud/  
  aws/  
  azure/  
09_iocs/  
10_reports/
```

3.4 Windows analysis host

Use a supported 64-bit Windows release, a pinned Python environment or official standalone Volatility executable, PowerShell 7 where possible, sufficient local SSD space and an offline symbols cache. Record the hashes of analysis tools. Disable automatic execution from evidence media and exclude the evidence repository from indexing if policy permits.

3.5 Linux analysis host

Linux is often the most convenient host for automation, package isolation, YARA, reverse engineering and large-scale text processing. Use Python virtual environments, separate analyst and malware-execution accounts, and read-only mounts for original evidence.

3.6 macOS analyst host

A macOS host can run Volatility and supporting tools, but modern macOS memory acquisition and Volatility's macOS analysis support are limited. The official Volatility documentation states that macOS support is no longer actively maintained. Treat macOS workflows as version-specific and validate every plugin against the exact image.^[6]

3.7 Virtual-machine controls

For malware research, hypervisor snapshots are usually superior to live acquisition because they are repeatable and can preserve a more coherent state. However, snapshot files are hypervisor-specific and may require conversion or supporting metadata. Preserve the complete VM directory, not only the apparent memory file.

3.8 Isolated networking

Use one of three modes:

Mode	Use	Risk control
No network	Static analysis and high-risk unknown samples	Strongest containment; no C2 behavior observed.
Host-only	Controlled service simulation and packet capture	No route to production or internet; analyst-controlled DNS/HTTP.

Filtered egress	Authorized dynamic analysis requiring limited external behavior	Explicit allowlists, full capture, sink-hole/proxy, rapid kill switch.
-----------------	---	--

Free and Verified Memory Acquisition

4.1 Acquisition strategy

Choose the method that preserves the evidence required by the investigative question with the lowest acceptable operational risk. The best tool is not universal. Windows 11 security controls, Linux kernel lockdown, hypervisor configuration, cloud-provider boundaries and Apple Silicon all affect feasibility.

LOOK FIRST AT

Determine whether the target is physical or virtual, whether the OS is still responsive, whether shutdown is acceptable, how much RAM is installed, whether disk space is sufficient, whether encryption keys may be lost on shutdown, and whether the acquisition tool is already staged and hashed.

THEN LOOK AT

Record the tool version and executable hash, capture start/end times, note expected acquisition artifacts, hash the output, validate the image with Volatility and preserve the original read-only.

4.2 Windows: WinPmem

WinPmem is an open-source Windows physical-memory acquisition tool that produces RAW images with its standalone mini executable. The official project documents the command below. Its README explicitly lists Windows 7 through Windows 10; Windows 11 use must therefore be validated on the exact build and hardware rather than assumed.^[7]

```
winpmem_mini_x64.exe physmem.raw  
certutil -hashfile physmem.raw SHA256
```

Advantages: self-contained, fast to deploy, RAW output directly usable by Volatility.

Limitations: kernel driver footprint, live-memory smearing, potential incompatibility with modern security controls, no atomic snapshot.

Use when: the system is responsive, volatile evidence matters, and the exact build has been tested.

Avoid when: driver loading is prohibited, the system is critically unstable, or native crash/snapshot acquisition is safer.

4.3 Windows: complete crash dump

A configured Windows complete memory dump records the contents of system memory when the operating system crashes. Microsoft documents manual generation through keyboard, NMI or tools such as NotMyFault, with operational cautions and page-file requirements.^[8, 9]

A crash dump is often more internally coherent than a long live acquisition, but it is destructive to availability and still changes a small amount of state during crash handling.

CAUTION

Do not trigger a crash on a production system without explicit authorization and a tested recovery plan. A complete dump can be very large, can prolong outage time and may fail if page-file or destination requirements are not met.

4.4 Linux: AVML

AVML is an x86-64 userland acquisition tool written in Rust. Current AVML uses subcommands and can produce uncompressed LiME-format output or compressed output. The official syntax is:[10]

```
sudo ./avml acquire output.lime
sudo ./avml acquire --compress output.lime.compressed
sha256sum output.lime > output.lime.sha256
```

AVML tries `/dev/crash`, `/proc/kcore` and `/dev/mem` when a source is not specified. Kernel lockdown can prevent acquisition. Test the exact distribution and kernel; a partially written file after an error is not automatically valid.

4.5 Linux: LiME

LiME is a loadable kernel module that acquires memory in LiME, padded or raw format. It can write to a local file or stream over TCP. Because it requires compiling/loading a kernel module, it has a larger operational and evidentiary footprint than a prebuilt userland tool, but it can be effective when kernel symbols and headers are available.

A representative local-file invocation after building the module is:

```
sudo insmod lime.ko "path=/mnt/evidence/host01.lime format=lime"
sha256sum /mnt/evidence/host01.lime
sudo rmmod lime
```

Always verify parameter names against the module's current README and built version. Prefer LiME format over raw where supported because LiME preserves physical ranges.

4.6 Linux: kdump and vmcore

Kdump uses kexec to boot a reserved dump-capture kernel after a panic. The crashed kernel's memory is exposed through `/proc/vmcore` as ELF. The Linux kernel documentation notes that `makedumpfile -d 31` writes only kernel data; that filtering is unsuitable when userland malware, injected code or encryption material matters.[11]

```
# On a prepared system after kdump has captured the crash:
sudo cp /proc/vmcore /mnt/evidence/vmcore
sha256sum /mnt/evidence/vmcore > /mnt/evidence/vmcore.sha256
```

4.7 Virtual machines

For QEMU/KVM, use documented monitor/QMP memory-dump functionality and preserve VM metadata. For VirtualBox, use the debug VM core-dump capability. For VMware and Hyper-V,

preserve the full saved-state or snapshot set and document the product/version. Never assume that renaming a hypervisor file creates a valid RAW image.

4.8 Hardware DMA

Authorized DMA acquisition through PCIe/FPGA hardware can reduce dependence on the target OS, but modern IOMMU, Kernel DMA Protection, VBS and platform firmware can block or alter access. DMA is a specialist laboratory capability, not a universal solution. Use read-only acquisition features only; do not include memory-writing or code-execution functionality in an evidence workflow.

4.9 macOS acquisition reality

Volatility does not acquire memory. Its documentation points to `osxpmem` as an example, but also warns that macOS analysis support is no longer actively maintained. On modern Apple Silicon, there is no reliable universal free workflow equivalent to Windows RAW acquisition. Prefer controlled VM research, platform diagnostics, endpoint telemetry, unified logs, process samples and disk artifacts. Any full-memory workflow must be validated against the exact hardware and OS build.[\[6\]](#)

4.10 Cloud workloads

AWS EBS snapshots and Azure managed-disk snapshots preserve disk state, not guest RAM. Full memory normally requires an in-guest acquisition tool, an OS-native crash dump or provider-assisted capabilities. AWS Security Incident Response's 2026 EC2 Triage capability can collect volatile triage data through Systems Manager, including processes, network connections and logged-in users, but it is not described as a full physical-memory image.[\[12\]](#)

Volatility3 Installation and Environment

5.1 Version pinning

Record the exact framework, Python and plugin versions used in every case. Stable commands can change namespace or options. The official release page identifies version 2.28.0 as the latest stable release at publication time.^[1]

5.1.1 Windows PowerShell

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install volatility3==2.28.0
python -m volatility3.cli --version
```

If using the official standalone executable:

```
vol.exe --version
vol.exe -h > evidence\volatility_help.txt
Get-FileHash .\vol.exe -Algorithm SHA256
```

5.1.2 Linux and macOS host

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install volatility3==2.28.0
python -m volatility3.cli --version
```

5.2 Source installation

```
git clone https://github.com/volatilityfoundation/volatility3.git
cd volatility3
git checkout v2.28.0
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
python vol.py --version
```

Record the checked-out commit and do not silently update during a case.

5.3 Offline installation

On a connected staging host with the same platform and Python version:

```
python -m pip download --dest wheelhouse volatility3==2.28.0
python -m pip freeze > requirements-staging.txt
```

On the offline host:

```
python -m pip install --no-index --find-links wheelhouse volatility3==2.28.0
```

Hash and transfer the wheelhouse through approved media.

5.4 Windows symbols

Windows symbols are normally downloaded and cached automatically when internet access is available. For offline work, pre-populate the symbol cache on a connected system using representative images, then transfer the symbol directory with hashes. A successful `windows.info.Info` result should show the selected symbol file, kernel base, DTB, architecture and build information.

5.5 Linux and macOS symbols

Linux and macOS require an Intermediate Symbol File (ISF) matching the exact kernel. `dwarf2json` converts ELF/DWARF symbol and type data into Volatility's JSON format. The official project is the authoritative source.[\[13, 14\]](#)

Representative Linux command:

```
./dwarf2json linux --elf /usr/lib/debug/boot/vmlinux-$(uname -r) \
> linux-$(uname -r).json
xz -9 linux-$(uname -r).json
```

Place the ISF under the Volatility symbols directory. Exact paths depend on the installation type. Verify discovery with `isfinfo.IsfInfo`.

5.6 Plugin namespace and CLI errors

The CLI accepts identifiers such as:

```
vol.exe -f memory.raw windows.malware.suspicious_threads.SuspiciousThreads
```

It does *not* accept Python import paths beginning with `volatility3.plugins..` If the CLI reports “invalid choice,” copy the exact plugin name from `vol.exe -h`. Some plugins have deprecated aliases outside the `windows.malware` namespace; use the current namespace in new scripts.

5.7 Output renderers and logging

Use machine-readable output for comparison and automation:

```
vol.exe -q -f memory.raw -r csv windows.pslist.PsList \
> results\01_pslist.csv 2> results\01_pslist.err.txt
```

Separate standard output from standard error. An error file containing only a framework banner is not analytically useful; preserve files containing warnings, unsatisfied requirements, tracebacks or symbol

errors.

5.8 Third-party plugins

Treat third-party plugins as code introduced into the analysis environment. Pin the repository commit, review source, hash the files, document dependencies and validate output against known images. Never mix unreviewed plugins into a legally sensitive case without recording that decision.

Memory Dump Preparation and Validation

6.1 Preserve the original

6.1.1 PowerShell

```
New-Item -ItemType Directory -Force .\01_original, .\02_working, .\04_volatility
Get-FileHash .\01_original\host01.raw -Algorithm SHA256 |
    Format-List | Out-File .\01_original\host01.raw.sha256.txt
Copy-Item .\01_original\host01.raw .\02_working\host01_working.raw
```

6.1.2 Bash

```
mkdir -p 01_original 02_working 04_volatility
sha256sum 01_original/host01.raw | tee 01_original/host01.raw.sha256
cp --reflink=auto 01_original/host01.raw 02_working/host01_working.raw
chmod a-w 01_original/host01.raw
```

6.2 Identify the format

Use acquisition records first. File extensions are not authoritative. Supplement with:

```
file memory.img
xxd -l 128 memory.img
vol.exe -f memory.img banners.Banners
```

Crash dumps, LiME images, ELF vmcores and VM snapshots contain structural metadata that can be lost by naive conversion. Preserve the original format even when a working conversion is required.

6.3 Initial validation

For Windows:

```
vol.exe -f memory.raw windows.info.Info
vol.exe -f memory.raw windows.kpcrs.KPCRs
vol.exe -f memory.raw windows.statistics.Statistics
```

For Linux/macOS:

```
python vol.py -f memory.lime banners.Banners
python vol.py -f memory.lime linux.boottime.Boottime
python vol.py -f memory.raw mac.pslist.PsList
```

Validation criteria include coherent architecture, kernel identity, symbol selection, processor count, system time, plausible process output and absence of fatal layer errors.

6.4 Recognizing corruption and smearing

Indicators include:

- impossible PIDs, thread counts, dates or pointer values;
- processes active in one structure and terminated in another;
- acquisition process creation at the same time as the reported system time;
- objects that appear only in pool scans with code-like values in structural fields;
- plugins ending early or producing truncated trees.

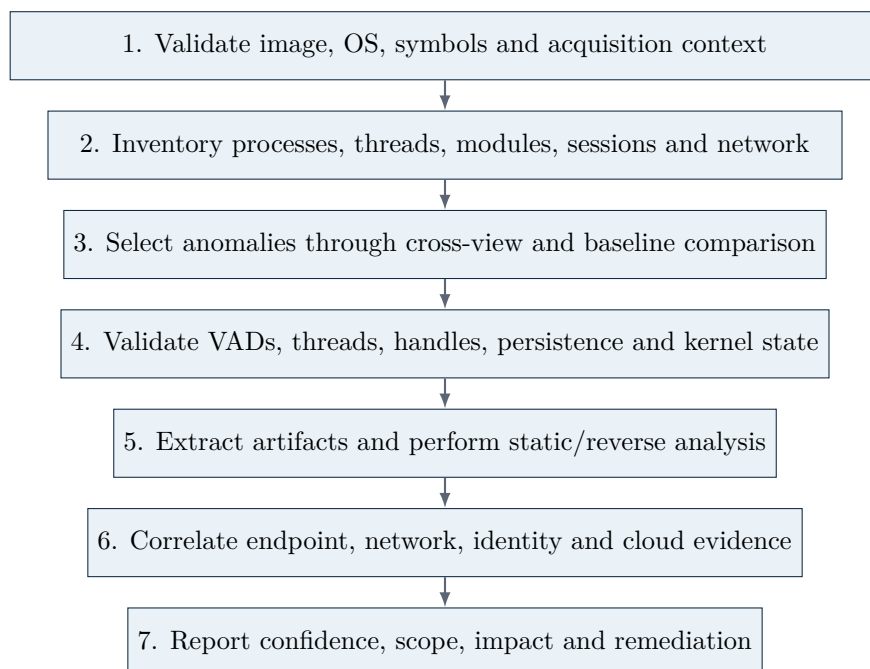
Corruption does not make an entire image useless. It changes confidence and requires cross-view validation.

6.5 Case manifest

Field	Example
Case ID	CASE-2026-0042
Evidence ID	MEM-HOST01-001
Source	HOST01 / serial / cloud resource ID
Acquisition method	WinPmem, complete dump, AVML, LiME, vmcore, VM snapshot
Tool version/hash	Version and SHA-256
Start/end UTC	ISO 8601 timestamps
Original SHA-256	Full hash
Working-copy SHA-256	Full hash
Known limitations	Smearing, missing pages, symbol mismatch, truncated source
Analyst	Named authorized examiner

Professional Analysis Methodology

7.1 The evidence funnel



7.2 Phase 0: define hypotheses

Examples:

- A userland process was injected and communicated with external infrastructure.
- Ransomware staging occurred through remote administration and credential theft.
- A kernel driver hid a process or redirected callbacks.
- A cloud identity was compromised and used to alter snapshots, keys or logging.

A hypothesis prevents random plugin execution. It also makes negative evidence meaningful.

7.3 Phase 1: establish system identity

LOOK FIRST AT

OS/kernel version, architecture, system time, boot time, processor count, symbol file, acquisition process and capture duration.

THEN LOOK AT

Clock skew, timezone, virtualization indicators, security products, acquisition artifacts and signs of corruption.

7.4 Phase 2: process inventory

Compare active-list, pool-scan and cross-view results. Build a table containing PID, PPID, image name, creation/exit time, path, command line, session, user/SID, token privileges and network endpoints.

7.5 Phase 3: ancestry and execution context

Prioritize:

- office/browser/email client spawning script engines or LOLBins;
- services spawning shells or user-profile executables;
- authentication processes with unexpected children;
- parent processes that terminated before child creation;
- system processes running from user-writable paths;
- duplicated Windows names with wrong paths or case-independent mismatch.

7.6 Phase 4: network mapping

Map every external or listening endpoint to PID, process, ancestry, user, command line and process creation time. Distinguish active endpoints from residual pool artifacts.

7.7 Phase 5: userland depth

Inspect loader lists, VADs, executable private memory, thread start addresses, hollowing/ghosting indicators, direct system calls, ETW patching, handles and extracted code.

7.8 Phase 6: persistence

Examine services, scheduled tasks, registry hives, Amcache, Shimcache, UserAssist, startup paths and cloud automation. Memory may contain only a partial persistence picture; correlate with disk and configuration logs.

7.9 Phase 7: kernel

Compare modules and pool scans, map driver objects to modules, validate IRP handlers, callbacks, SSDT, timers and kernel threads. A rootkit finding normally requires multiple independent kernel anomalies.

7.10 Phase 8: timeline and correlation

Normalize UTC, identify uncertainty caused by acquisition, and combine process, network, file, registry, endpoint and cloud events. Do not create precision that the source does not support.

7.11 Phase 9: reporting

Every finding should include:

1. observation;
2. evidence source and command;
3. interpretation;
4. alternative explanations considered;
5. confidence;
6. impact;
7. recommended next action.

7.12 Escalation matrix

Observation	Initial assessment	Escalation condition
<code>malfind</code> region only	Suspicious/low confidence	Thread starts inside region, PE/shellcode present, network or YARA correlation.
Process only in <code>psscan</code>	Often terminated/stale	No exit time, active threads/network, cross-view hiding.
Driver only in <code>modscan</code>	Unloaded/stale/false positive possible	Active driver object, callback, IRP or thread maps to it.
Unknown listening port	Needs owner mapping	Unexpected binary/path/user, external bind, persistence or exploit evidence.
Direct syscalls	Product/runtime behavior possible	Anonymous executable VAD, suspicious process context, unhooking and injection correlation.
Cloud API anomaly	Misconfiguration/admin activity possible	Compromised identity, impossible travel, disabled logging, key/snapshot deletion or exfiltration.

Windows Runtime Structures for Analysts

8.1 Why structure knowledge matters

Volatility does not search for process names in plain text and call the result a process list. Plugins reconstruct operating-system objects and relationships from kernel and process structures. Understanding what each view represents prevents false conclusions.

8.2 Processes and threads

The executive process object represents a process in kernel memory. Active-list enumeration follows the operating system's linked list, while pool scanning searches memory for object signatures. Thread objects contain creation/exit information and start addresses. A process can disappear from an active list because it terminated, because the list is smeared, or because it was intentionally unlinked. The distinction requires cross-view evidence.

8.3 PEB, loader lists and VADs

The Process Environment Block (PEB) contains user-mode metadata including image path, command line and loader lists. A process can manipulate PEB strings, and protected or corrupted processes may have unreadable PEB data. Loader-list plugins compare modules represented in different PEB lists with mapped executable regions.

Virtual Address Descriptors describe ranges in a process address space. Important attributes include start/end address, page protection, private versus mapped memory and backing file. Executable private memory is a lead, not a verdict. JIT compilers, browser sandboxes, security software and runtime frameworks legitimately create executable private pages.

8.4 Handles and objects

A process handle table can expose file, registry key, process, thread, token, section, mutant, semaphore and event objects. Ransomware and remote tooling can leave useful file or named-object handles, but a handle proves access, not malicious intent or specific I/O activity.

8.5 Kernel modules, driver objects and callbacks

A loaded kernel module is an image mapped into kernel address space. A driver object contains dispatch pointers and device relationships. These concepts are related but not identical. Rootkit analysis therefore compares module lists, scanned module structures, driver objects, IRP handlers, callbacks, threads and timers.

8.6 Security context

Tokens, SIDs, integrity, sessions and privileges provide execution context. A SYSTEM process with `SeDebugPrivilege` is not automatically malicious. A user application unexpectedly running as SYSTEM, or a service binary from a user-writable path with powerful privileges, is materially more significant.

8.7 Protected processes and VBS

Protected Process Light, Credential Guard, Virtualization-Based Security and isolated user mode can reduce visibility or alter expected structures. Failure to read a PEB or module list in a protected process is not, by itself, evidence of hiding. Record platform security settings and compare affected processes with expected Windows behavior.

Userland Malware Analysis

9.1 Process inventory and cross-view analysis

LOOK FIRST AT

Run `windows.pslist.PsList`, `windows.psscan.PsScan`, `windows.pstree.PsTree`, `windows.malware.psxview.PsXView` and `windows.cmdline.CmdLine`. Establish active, terminated, hidden-candidate and corrupted objects before examining injection.

```
vol.exe -q -f memory.raw -r csv windows.pslist.PsList > 01_pslist.csv
vol.exe -q -f memory.raw -r csv windows.psscan.PsScan > 02_psscan.csv
vol.exe -q -f memory.raw -r csv windows.pstree.PsTree > 03_pstree.csv
vol.exe -q -f memory.raw -r csv windows.malware.psxview.PsXView > 04_psxview.csv
vol.exe -q -f memory.raw -r csv windows.cmdline.CmdLine > 05_cmdline.csv
```

9.1.1 How to read the output

Build differences by PID and object offset. A process recovered by `psscan` but not `pslist` is often terminated; check `ExitTime`. A stronger hidden-process candidate is absent from the active list, has no exit time, and still has live threads, handles or network artifacts. Cross-view flags are not independent proof because several methods may depend on related structures.

9.1.2 Suspicious indicators

- system image name with a path outside `System32` or the expected Windows directory;
- impossible or unusual parent-child relationship;
- process created immediately before the incident with no legitimate application context;
- encoded script command lines, remote URLs, archive extraction, shadow-copy deletion or credential tooling;
- process name differing from the real image path;
- live process absent from expected enumeration views.

9.1.3 Common mistakes

Do not mark every `svchost.exe`, `rundll32.exe`, `powershell.exe` or `msedgewebview2.exe` as malicious. These are execution hosts. The command line, parent, path, loaded content, user context and behavior determine risk.

Next command: `windows.getsids.GetSIDs,windows.privileges.Privs,windows.sessions.Sessions`

9.2 Hierarchy and command-line analysis

The process tree is one of the highest-value triage views because malware commonly relies on legitimate parents and child processes. Investigate both directions: who created the suspicious process, and what did it create?

Pattern	Investigative interpretation
Office application → script engine	Possible malicious document, add-in or user-driven automation; inspect command line and files.
Browser → shell/LOLBin	Possible download execution, browser exploit or user action.
Service host → user-profile binary	Service persistence or updater; verify service configuration and signature.
Authentication process → unusual child	High priority; validate protected-process behavior and injection.
Shell → vssadmin/wbadmin/bcdedit	Common ransomware preparation or administration; correlate timing and account.
Remote-management agent → shell	Could be legitimate administration or intrusion; verify ticket/change record and source.

9.3 Security identifiers, privileges and sessions

```
vol.exe -f memory.raw windows.getsids.GetSIDs
vol.exe -f memory.raw windows.privileges.Privs
vol.exe -f memory.raw windows.sessions.Sessions
vol.exe -f memory.raw windows.joblinks.JobLinks
```

Look for user applications with service identities, unexpected interactive sessions, privilege enablement that does not match process purpose, and processes attached to unusual jobs. Security products and administrative agents can legitimately hold powerful privileges.

9.4 DLL and module analysis

```
vol.exe -q -f memory.raw -r csv windows.dlllist.DllList > dlllist.csv
vol.exe -q -f memory.raw -r csv windows.malware.ldrmodules.LdrModules > ldrmodules.csv
vol.exe -q -f memory.raw -r csv windows.verinfo.VerInfo > verinfo.csv
```

`DllList` walks process loader information. `LdrModules` compares mapped executable regions against loader-list membership. Prioritize executable images that are mapped but absent from all loader lists, especially from user-writable paths or with active threads inside them.

False positives include the main executable, resource-only mappings, .NET assemblies, protected processes, partially paged structures and terminated processes. A loader discrepancy becomes significant when the mapping is executable code, has no legitimate backing path, or is referenced by a thread.

9.5 VAD analysis

```
vol.exe -f memory.raw windows.vadinfo.VadInfo --pid 4321
vol.exe -f memory.raw windows.vadwalk.VadWalk --pid 4321
vol.exe -f memory.raw windows.memmap.Memmap --pid 4321
```

Read VADs in context:

- **PAGE_READWRITE**: data or heap; not executable in the reported state.
- **PAGE_EXECUTE_READ**: executable but not writable; common for code mappings and generated code after protection changes.
- **PAGE_EXECUTE_READWRITE**: higher-interest, but common in JIT and some security products.
- **Private memory + executable + no file**: potential injected/generated code.
- **Mapped image with wrong or missing path**: possible unlinking, deleted file or corruption.

9.6 Injected-code detection with malfind

```
vol.exe -q -f memory.raw -r csv windows.malware.malfind.Malfind > malfind.csv
vol.exe -o dumps\4321 -f memory.raw windows.malware.malfind.Malfind \
--pid 4321 --dump
```

Malfind selects regions with characteristics associated with injected or unpacked code. Interpret the hexdump and disassembly, then correlate region bounds with thread start addresses and module paths.

DECISION RULE

Do not report “malware found” from **malfind** alone. Escalate when the same region contains a PE header or coherent shellcode, is not a known JIT/runtime allocation, is referenced by a thread, matches a rule, or belongs to a process with suspicious ancestry/network activity.

9.7 Thread analysis

```
vol.exe -q -f memory.raw -r csv windows.threads.Threads > threads.csv
vol.exe -q -f memory.raw -r csv windows.thrdscan.ThrdScan > thrdscan.csv
vol.exe -q -f memory.raw -r csv \
windows.malware.suspicious_threads.SuspiciousThreads > suspicious_threads.csv
vol.exe -q -f memory.raw -r csv \
windows.suspended_threads.SuspendedThreads > suspended_threads.csv
```

The real **SuspiciousThreads** output is selective. A general thread listing is not a list of suspicious threads. Confirm the plugin name from local help and check that columns match the documented plugin output.

High-value observations include:

- start or Win32 start address in anonymous/private executable memory;
- address outside all known modules;
- context instruction pointer inside a suspicious VAD;
- suspended thread in a newly created process combined with hollowing indicators;
- kernel thread start address outside loaded modules.

9.8 Hollowing, ghosting and PEB masquerading

```
vol.exe -f memory.raw windows.malware.hollowprocesses.HollowProcesses
vol.exe -f memory.raw windows.malware.processghosting.ProcessGhosting
vol.exe -f memory.raw windows.malware.pebmasquerade.PebMasquerade
```

These plugins detect structural inconsistencies associated with specific techniques. They can miss variants and can generate false positives from case-only path differences, protected processes, deleted files or corruption. Validate the process image, PEB path, command line, VADs, loader state and threads.

9.9 Direct, indirect and unhooked system calls

```
vol.exe -f memory.raw windows.malware.direct_system_calls.DirectSystemCalls
vol.exe -f memory.raw windows.malware.indirect_system_calls.IndirectSystemCalls
vol.exe -f memory.raw windows.malware.unhooked_system_calls.UnhookedSystemCalls
```

These plugins identify code patterns and alternate system-call implementations. Browsers, sandboxes, security software and compatibility components may generate systematic tables. A regular table repeated in multiple related processes is materially different from a small anonymous stub in a suspicious process. Determine VAD protection, backing path, repetition, process role and execution evidence.

9.10 ETW patching and import analysis

```
vol.exe -f memory.raw windows.etwpatch.EtwPatch
vol.exe -f memory.raw windows.iat.IAT --pid 4321
```

A patch finding requires validation against the expected module version and memory acquisition quality. Endpoint security products may hook or instrument functions legitimately. Report the exact function, module, original/observed bytes, process and supporting behavior.

9.11 Handles, mutants and file objects

```
vol.exe -f memory.raw windows.handles.Handles --pid 4321
vol.exe -f memory.raw windows.mutantscan.MutantScan
vol.exe -f memory.raw windows.filescan.FileScan
```

Handles can reveal open ransom notes, staged archives, named pipes, process access, registry keys and synchronization objects. A mutex-like object can support malware-family identification, but names are easily changed and collisions occur.

9.12 Environment variables

```
vol.exe -f memory.raw windows.envvars.Envvars --pid 4321
```

Environment variables can reveal user profiles, temp paths, execution context, proxy configuration and staging directories. They are not proof that the process used those values.

9.13 YARA scanning

```
vol.exe -f memory.raw windows.vadyarascan.VadYaraScan \  
--pid 4321 --yara-file rules.yar  
vol.exe -f memory.raw yarascan.YaraScan --yara-file rules.yar
```

Start with narrow, incident-specific rules. Large public rule sets produce high noise and can disclose sensitive data through matches. Record rule version and hash. YARA confirms a byte-pattern match, not the complete behavior or attribution.

9.14 Extraction and reverse-engineering handoff

```
mkdir dumps\4321  
vol.exe -o dumps\4321 -f memory.raw windows.pslist.PsList --pid 4321 --dump  
vol.exe -o dumps\4321 -f memory.raw windows.dllexport.DllList --pid 4321 --dump  
vol.exe -o dumps\4321 -f memory.raw windows.vadinfo.VadInfo \  
--pid 4321 --address 0x7ff600001000 --dump
```

Hash extracted files, record source PID/address, and analyze copies. Memory-dumped PE files may have damaged headers, reconstructed sections or missing imports. Use Ghidra, FLOSS, capa, YARA and a PE parser as follow-up, clearly labeling reconstructed artifacts.

9.15 Credential artifact awareness

Version 2.28.0 includes plugins such as `windows.hashdump.Hashdump`, `windows.cachedump.Cachedump`, `windows.lsadump.Lsadump` and `windows.malware.skeleton_key_check.Skeleton_Key_Check`. Their output is highly sensitive. Use only when scope authorizes credential investigation, store output separately, and focus reporting on exposure and reset requirements. This manual does not include password-cracking procedures.

Network Artifact Analysis

10.1 Objectives and limitations

Memory can preserve sockets, endpoints, local/remote addresses, ports, states, owning PIDs and timestamps. It does not replace packet capture, DNS logs, proxy logs, flow records or TLS metadata. Endpoint structures can persist after connection closure, and pool scanning may recover stale objects.

LOOK FIRST AT

External established connections, unusual listening ports, remote-management ports, public IPs, connections owned by unexpected processes, and endpoints created near suspicious process activity.

THEN LOOK AT

Process ancestry, command line, user, modules, injected regions, DNS/proxy/firewall evidence, destination reputation, byte counts and cloud flow logs.

10.2 Windows commands

```
vol.exe -q -f memory.raw -r csv windows.netscan.NetScan > netscan.csv  
vol.exe -q -f memory.raw -r csv windows.netstat.NetStat > netstat.csv
```

If one plugin is empty and the other returns data, do not conclude that the system had no network activity. The plugins rely on different mechanisms and build support. Document which output is usable.

10.3 Interpretation order

1. Identify state: established, listening, closed/residual or UDP.
2. Identify local binding: loopback, specific interface or all interfaces.
3. Map PID to process name, path and creation time.
4. Determine whether the PID was active or terminated.
5. Classify destination: internal, private, public, multicast, broadcast or loopback.
6. Correlate with process memory and external logs.

10.4 Listening ports

A listening port is exposure, not compromise. Validate service ownership, expected binding and firewall policy. A legitimate service listening on 0.0.0.0 may unnecessarily expand attack surface. A high random port owned by a signed remote-management agent can be normal; the same port owned by a user-profile binary is higher risk.

10.5 C2 indicators

Potential C2 patterns include periodic outbound connections, rare destinations, unusual ports, direct IP connections, domain-generation behavior, encrypted sessions from unexpected binaries, or connections from injected processes. Memory normally provides only a point-in-time view; periodicity and volume require flow/proxy/packet telemetry.

10.6 DNS limitations

Volatility's general Windows network plugins do not reconstruct a complete DNS history. Search process strings and cache-related memory only as leads, then validate with DNS resolver, endpoint, browser, proxy or cloud logs. A domain string in memory does not prove a successful resolution or connection.

10.7 Linux network analysis

```
python vol.py -f memory.lime linux.ip.Addr
python vol.py -f memory.lime linux.ip.Link
python vol.py -f memory.lime linux.sockstat.Sockstat
python vol.py -f memory.lime linux.sockscan.Sockscan
```

Interpret network namespaces. A socket in a container namespace may map to a host PID or interface differently than expected. Record namespace identifiers, process mappings and container runtime metadata.

10.8 macOS network analysis

```
python vol.py -f memory.raw mac.netstat.Netstat
python vol.py -f memory.raw mac.ifconfig.Ifconfig
python vol.py -f memory.raw mac.socket_filters.Socket_filters
```

Results are version-dependent because macOS support is not actively maintained. Cross-check with unified logs, endpoint telemetry and packet/flow data.

10.9 Professional example

FIELD NOTE

Illustrative synthetic finding: `powershell.exe` PID 4108, created by `winword.exe`, held an established TCP endpoint to a public address. The command line downloaded a script, `malfind` identified a private executable region, and a thread began inside that region. The network endpoint alone was low confidence; the correlated process and memory evidence supported a probable userland compromise.

Ransomware Investigation and Response

11.1 Why ransomware requires a separate workflow

Ransomware operations are multi-stage intrusions. Encryption is often the last visible action after days or weeks of identity compromise, reconnaissance, credential theft, lateral movement, backup disruption and data exfiltration. The responder must therefore investigate both the encryptor and the intrusion that enabled it.

The FBI's 2025 IC3 report recorded more than 3,600 ransomware complaints and reported losses exceeding 32 million dollars, while warning that the reported figures exclude many business-interruption and remediation costs and do not represent all incidents. NIST's 2026 ransomware profile treats ransomware risk management as a combination of governance, preparation, detection, response, recovery and communication.[15, 4]

11.2 The first-hour ransomware priorities

LOOK FIRST AT

Human safety, destructive activity, affected identity accounts, domain controllers, hypervisors, backup systems, cloud control planes, active encryption, exfiltration, and the business services at risk.

THEN LOOK AT

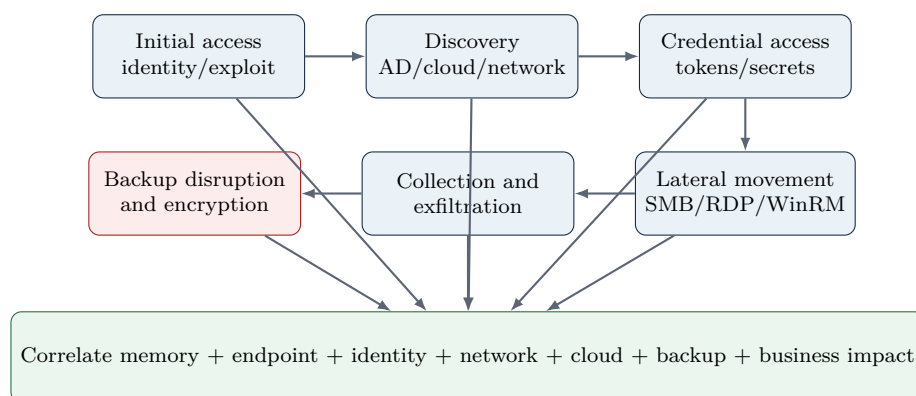
Volatile memory, process trees, command lines, network connections, EDR telemetry, authentication logs, disk snapshots, ransom notes, encrypted samples, cloud API history, backup integrity and initial-access evidence.

1. Activate incident command, legal/privacy and executive contacts.
2. Isolate affected systems with EDR or network controls where safe; avoid mass power-off without a plan.
3. Preserve memory from representative systems: encryptor host, initial-access host, domain controller, file server, backup server and management jump host.
4. Snapshot disks and cloud volumes; preserve hypervisor and backup logs.
5. Disable or restrict confirmed compromised identities and revoke sessions/tokens.
6. Preserve firewall, DNS, proxy, VPN, identity, EDR and cloud audit logs.
7. Identify the encryption start time and work backward.

8. Contact appropriate law enforcement and national response authorities according to jurisdiction and policy.

CISA's StopRansomware guidance recommends isolating impacted systems, triaging restoration and recovery, and preserving evidence while coordinating external reporting. The FBI does not support paying a ransom because payment does not guarantee recovery and encourages the ecosystem.[16, 17]

11.3 Ransomware evidence model



11.4 Memory-forensics objectives

11.4.1 Identify the encryptor and launch chain

Use process lists, tree, command lines, sessions, SIDs and privileges. Search for mass-execution tooling, script engines, service-created binaries and processes launched by remote-management agents.

```

vol.exe -f memory.raw windows.pstree.PsTree
vol.exe -f memory.raw windows.cmdline.CmdLine
vol.exe -f memory.raw windows.getsids.GetSIDs
vol.exe -f memory.raw windows.privileges.Privs
vol.exe -f memory.raw windows.svcscan.SvcScan
  
```

Priority command-line strings include backup/shadow-copy administration, event-log clearing, firewall modification, remote execution, archive tools, cloud-storage clients and mass file operations. Such commands can be legitimate; correlate with account, timing and change records.

11.4.2 Find unpacked code and configuration

Ransomware samples may unpack into memory or inject into another process. Run `malfind`, thread analysis, VAD YARA scanning and extraction. Search dumps for ransom-note names, extensions, campaign identifiers, mutex names, configuration, public keys and command-and-control data.

11.4.3 Encryption-key awareness

Runtime memory may contain symmetric keys, key schedules or cryptographic context. Preserving memory before shutdown can be valuable, but key recovery is malware-family and implementation specific. Do not promise decryption. Coordinate with law enforcement, trusted decryptor projects and specialist reverse engineers. Maintain strict handling because the same memory can contain credentials and private data.

11.4.4 Lateral movement

Look for:

- remote service creation and service binaries;
- RDP, SMB, WinRM, SSH or remote-management processes and endpoints;
- command shells under management agents;
- authentication sessions and elevated tokens;
- handles to remote shares or administrative tools;
- domain-controller and identity-provider events outside memory.

11.4.5 Backup and recovery disruption

Memory can show commands and processes used to alter backups, but definitive evidence usually resides in backup-platform audit logs, Windows event logs, hypervisor logs and cloud control-plane records. Preserve those logs immediately because ransomware actors frequently target backup administration.

11.5 Endpoint-forensics objectives

Collect and correlate:

- EDR process, network, file and registry telemetry;
- Windows Security 4688 process creation where enabled;
- System 7045 service installation;
- Security 4698 scheduled task creation where audited;
- Sysmon process, network, image-load, process-access, file and DNS events according to the deployed configuration;
- Prefetch, Amcache, Shimcache, MFT, USN Journal and event logs from disk;
- antivirus/EDR detections and quarantine records;
- ransom notes and representative encrypted files;
- backup, virtualization and storage audit logs.

Memory timestamps and disk timestamps must be normalized to UTC and checked for clock skew. Do not infer execution from Shimcache alone; combine artifacts.

11.6 Network-forensics objectives

Work backward from encryption time to identify:

- initial remote access and VPN authentication;
- scanning and discovery across internal ranges;
- SMB/RDP/WinRM/SSH lateral movement;
- large outbound transfers, cloud-storage uploads or archive staging;
- C2 infrastructure and remote-management channels;
- traffic to backup, hypervisor and identity-management systems.

Packet captures provide payload and timing detail; NetFlow/VPC/NSG flow logs provide coverage and direction; proxy/DNS logs provide destination context. Memory supplies the process owner and runtime state.

11.7 Cloud ransomware

Cloud ransomware can involve guest OS encryption, object deletion/encryption, snapshot deletion, KMS/Key Vault misuse, backup-vault attacks, identity takeover or destructive infrastructure-as-code changes. The cloud control plane is often more important than the guest encryptor.

For AWS, preserve CloudTrail, GuardDuty, Security Hub findings, IAM changes, KMS events, S3 object/version activity, EBS snapshot actions and backup-vault activity. For Azure, preserve Entra sign-ins/audit logs, Azure Activity Logs, Defender for Cloud/Sentinel incidents, Key Vault logs, storage operations, snapshots and Azure Backup events.

11.8 Containment decision points

Decision	Benefit	Evidence/operational risk
Network isolate endpoint	Stops spread/exfiltration	Can break remote acquisition and volatile collection.
Disable identity	Stops reuse	May terminate sessions and alter evidence; preserve identity logs first where possible.
Power off/reboot	Stops encryption on that host	Loses RAM, keys and active connections; may trigger boot-time behavior.
Snapshot disks	Preserves point-in-time disk state	Snapshot is crash-consistent, not RAM; cloud API actions must be logged.
Stop cloud VM	Reduces activity/cost	Loses guest RAM; IP/storage behavior may change.
Block destination	Rapidly reduces C2/exfiltration	Actor may switch infrastructure; preserve DNS/proxy/flow data first.

11.9 Real incident lessons

11.9.1 Colonial Pipeline

The U.S. Department of Justice documented that Colonial Pipeline was attacked by DarkSide in May 2021 and reported the incident quickly to the FBI; portions of infrastructure were taken out of operation and a ransom of approximately 75 bitcoin was paid. The lesson is not a specific memory artifact. It is that ransomware response is a business-continuity and critical-infrastructure decision, and rapid external coordination can materially affect outcomes.[18]

11.9.2 Change Healthcare

UnitedHealth Group's SEC filing stated that it identified unauthorized access affecting Change Healthcare systems on 21 February 2024 and isolated impacted systems to contain, assess and remediate

the incident. The event demonstrates concentration risk and the need to map downstream business dependencies while technical containment is underway.[19]

11.9.3 ESXiArgs

CISA documented that ESXiArgs ransomware encrypted configuration files on vulnerable ESXi servers and could render VMs unusable. The investigation boundary must include the hypervisor, datastore, VM configuration, management plane, backups and guest evidence. A guest memory dump alone cannot explain a hypervisor-level ransomware event.[20]

11.10 Ransomware reporting

A ransomware report should separate:

- initial access and identity compromise;
- actor dwell time and discovery;
- lateral movement and privilege escalation;
- exfiltration evidence and affected data;
- backup/recovery impact;
- encryption scope and start time;
- affected business services and third parties;
- containment actions and evidence impact;
- recovery confidence and monitoring plan;
- notification and regulatory decisions owned by legal/privacy teams.

Ransomware Deep Technical Playbooks

12.1 Investigate the operation, not only the encryptor

The visible encryption process is usually the final impact component of a broader operation. A professional ransomware investigation creates separate workstreams for initial access, identity compromise, lateral movement, privilege escalation, command-and-control, staging, exfiltration, backup disruption, encryption, business impact and recovery assurance. These workstreams share a single evidence timeline but can have different owners and containment priorities.

LOOK FIRST AT

Identify whether encryption is active, which identities and management systems are compromised, whether backups are being altered, whether data is leaving the environment, and which systems control recovery. Preserve representative volatile evidence before reboot or deallocation when operationally safe.

THEN LOOK AT

Determine initial access, dwell time, lateral paths, administrator actions, exfiltration scope, encryptor deployment method, cloud API activity, affected data classes and recovery integrity.

12.2 Role-based incident organization

Use explicit operational roles:

Role	Primary responsibility	Evidence decisions
Incident commander	Priorities, safety, business decisions, external coordination	Approves disruptive containment and evidence tradeoffs
Forensic lead	Evidence plan, chain of custody, technical confidence	Selects representative hosts and acquisition sequence
Identity lead	AD, Entra, IAM, PAM, federation and token response	Preserves sign-in/audit logs before resets/revocation
Network lead	Segmentation, egress, flow and packet evidence	Balances blocking with capture and management access
Cloud lead	Accounts, subscriptions, snapshots, audit logs and roles	Prevents destructive API activity and preserves provider records
Recovery lead	Backups, restoration, clean-room rebuild and validation	Protects recovery systems from compromised identity/control paths

Legal/privacy	Notification, privilege, regulatory and data-impact guidance	Controls sensitive evidence distribution and retention
Communications	Internal/external messaging	Uses verified facts and avoids premature attribution

12.3 The representative-system acquisition plan

Capturing every host may be impossible. Prioritize systems that answer distinct questions:

System	Why it matters	High-value evidence
Earliest detected host	Initial foothold or first known activity	Browser/mail/remote access, process history, credentials, initial payload
Active encryptor host	Impact mechanism and run-time material	Encryptor, keys/config, file handles, threads, sockets, command line
Domain controller/identity server	Scope and privilege escalation	Sessions, authentication, directory changes, replication and service activity
Administrative jump host	Operator activity and tooling	Remote sessions, scripts, tokens, archives, management clients
Backup server/appliance	Recovery viability and destructive access	Admin sessions, jobs, retention changes, deletion events, credentials
Hypervisor manager	VM-level impact and deployment	API tasks, sessions, datastore changes, VM power/snapshot events
Cloud workload	Guest plus control-plane compromise	Memory, disks, instance metadata, role credentials, API and flow logs
File server	Encryption chronology and affected shares	Open handles, SMB sessions, MFT/USN, ransom notes, file extension patterns

12.4 Windows encryptor triage

12.4.1 Process and file-impact correlation

An encryptor commonly has many writable file handles, high I/O, worker threads and access to local or remote volumes. Volatility cannot calculate complete file-I/O rates from a single image, but handles and process context can identify a candidate.

```
vol.exe -q -f memory.raw -r csv windows.pstree.PsTree > pstree.csv
vol.exe -q -f memory.raw -r csv windows.cmdline.CmdLine > cmdline.csv
vol.exe -q -f memory.raw -r csv windows.handles.Handles > handles.csv
vol.exe -q -f memory.raw -r csv windows.threads.Threads > threads.csv
vol.exe -q -f memory.raw -r csv windows.netstat.NetStat > netstat.csv
vol.exe -q -f memory.raw -r csv windows.malware.malfind.Malfind > malfind.csv
```

Rank processes by:

- number and type of writable file handles;
- paths across multiple user shares or volumes;
- parent process and launch mechanism;

- creation time relative to first encrypted file;
- token privileges and user/session;
- remote endpoints and staging services;
- executable private memory and suspicious threads;
- service/task/remote-management relationships.

12.4.2 Command-line indicators for recovery inhibition

Search command lines and console artifacts for administrative utilities and parameters associated with shadow copies, backup catalogs, boot recovery, event logs, firewall rules and service control. Examples of strings to hunt include:

```
vssadmin ... shadows
wmic ... shadowcopy
wbadmin ... catalog
bcdedit ... recoveryenabled
wevtutil ... clear-log
sc.exe ... stop
powershell ... Get-WmiObject / Invoke-Command
```

These fragments are indicators, not proof. Backup software and administrators use the same tools. Validate account, parent, host role, change window, exact arguments and subsequent impact. Do not reproduce destructive commands during analysis.

12.4.3 Ransom-note and extension correlation

Search memory, file objects and disk evidence for the note filename, contact address, onion URL, victim ID, extension and campaign marker. A string in memory can originate from antivirus scanning, browser viewing or a ransom note opened by an analyst; establish process ownership and timing.

```
vol.exe -f memory.raw yarascan.YaraScan --yara-file ransomware_strings.yar
vol.exe -f memory.raw windows.vadyarascan.VadYaraScan \
--pid 6840 --yara-file ransomware_strings.yar
```

Verify the actual YARA plugin options with local help. Keep case-specific strings separate from public family rules.

12.4.4 Cryptographic material

Memory may contain:

- symmetric keys or expanded key schedules;
- public keys and victim-specific key material;
- cryptographic provider contexts;
- filenames currently being processed;
- in-memory configuration controlling exclusions, shares and extensions.

Recovery is not guaranteed. Key material may be ephemeral, transformed, protected by a remote service or already erased. Preserve the full process dump and exact VADs before attempting family-specific recovery. Coordinate with a qualified cryptanalyst and trusted decryptor resources.

12.5 Linux ransomware playbook

Linux ransomware can target application servers, NAS systems, databases, container nodes and virtualization platforms. The process may be a native ELF, script, statically linked binary, deleted executable or command received through a management agent.

12.5.1 First pass

```
python3 vol.py -q -f host.lime -r csv linux.psaux.PsAux > psaux.csv
python3 vol.py -q -f host.lime -r csv linux.pstree.PsTree > pstree.csv
python3 vol.py -q -f host.lime -r csv linux.lsof.Lsof > lsof.csv
python3 vol.py -q -f host.lime -r csv linux.proc.Maps > maps.csv
python3 vol.py -q -f host.lime -r csv linux.sockstat.Sockstat > sockets.csv
python3 vol.py -q -f host.lime -r csv linux.malware.malfind.Malfind > malfind.csv
python3 vol.py -q -f host.lime -r csv linux.mountinfo.MountInfo > mounts.csv
```

Look for executables under `/tmp`, `/var/tmp`, shared memory, deleted paths, unexpected static binaries, broad access to mounted storage, unusual parentage under SSH/systemd/container agents, and sockets to external infrastructure.

12.5.2 Systemd and scheduled execution

Memory can reveal process ancestry and open unit files, but validate persistence from disk:

- system and user systemd units/timers;
- cron directories and user crontabs;
- shell profiles and SSH keys;
- container manifests and orchestration jobs;
- cloud-init and management-agent histories.

12.5.3 Databases and application consistency

Encryption of database files while services are running can produce both malicious impact and application-level corruption. Preserve database logs, transaction logs, service memory and storage snapshots. Do not assume file-level restoration is sufficient; perform application-consistent recovery and integrity checks.

12.6 VMware ESXi and virtualization-platform ransomware

A hypervisor incident has different evidence priorities from guest ransomware. Preserve:

- management-server and hypervisor authentication logs;
- API/task/event history;
- shell and SSH activity;
- host configuration and enabled services;
- datastore metadata, VM configuration and snapshot state;
- management-plane identities and MFA events;
- backup-proxy and storage-array logs;

- representative encrypted and unencrypted VM files.

Guest memory is valuable only if the guest was running and capture is feasible. A powered-off or encrypted VM cannot provide a current guest RAM image. Hypervisor and management evidence can establish how encryption was deployed across many VMs.

CAUTION

Do not perform mass VM power operations or delete snapshots during triage. Snapshot-chain and datastore actions can destroy recovery options and alter chronology. Use virtualization and storage specialists.

12.7 Double-extortion investigation

Encryption does not prove exfiltration, and a ransom claim does not prove the stated volume. Establish exfiltration through evidence:

- archive creation and staging paths;
- file-access history and process handles;
- outbound byte counts and long-lived sessions;
- proxy, DNS, firewall, DLP and flow logs;
- cloud-storage client configuration and API logs;
- object-store access, snapshots, database exports and mailbox access;
- actor-provided sample data, validated without contacting infrastructure directly.

12.7.1 Volume estimation

Network flow byte counts are not identical to source-data size because of compression, encryption, retries and protocol overhead. Use a range and document assumptions. Correlate archives on disk, process memory, outbound flows and destination service logs.

12.8 Identity-first ransomware scoping

Investigate accounts in four groups:

1. initial-access identities;
2. privileged identities used for discovery/lateral movement;
3. service accounts and application identities exposed in memory or configuration;
4. recovery-system identities controlling backups, hypervisors and cloud infrastructure.

For each, preserve authentication, MFA, token, role, device and administrative-change history. Resetting a password may not revoke OAuth grants, cloud role sessions, API keys, certificates, Kerberos tickets or active remote sessions.

12.9 Active Directory evidence plan

Preserve domain-controller security and directory-service logs, replication changes, group membership, GPO changes, new services/tasks, delegation settings, trusts, certificate services and privileged access

systems. Memory from a domain controller can show active processes, modules, network sessions and credentials, but directory modification history usually requires logs and database-level forensics.

12.10 AWS ransomware event families

High-value CloudTrail searches include events related to:

- IAM policy, role, access key and trust changes;
- KMS key policy, grants, disablement and scheduled deletion;
- EBS snapshot creation, copy, sharing and deletion;
- AWS Backup vault/recovery-point changes;
- S3 bucket policy, versioning, replication, lifecycle and object actions where data events are enabled;
- CloudTrail trail changes and logging stop/delete actions;
- SSM commands and automation executions;
- Organizations and cross-account changes.

Use CloudTrail event names from the actual service documentation and collected events rather than maintaining an unverified static list in a playbook. Attackers and administrators call the same APIs; identity, sequence and business context matter.

12.11 Azure ransomware event families

High-value Azure/Entra searches include:

- Azure RBAC role assignments and custom role changes;
- Entra directory roles, app registrations, service-principal credentials and consent grants;
- managed disk/snapshot deletion and creation;
- Recovery Services/Backup vault policy and recovery-point changes;
- Key Vault RBAC/access-policy, key and secret operations;
- Storage account key, network, retention, versioning and object activity;
- diagnostic-setting deletion and log-routing changes;
- Run Command, extensions and automation actions on VMs.

12.12 Backup compromise assessment

A backup is trustworthy only when:

- the recovery point predates confirmed initial access or has been independently validated;
- backup management identities were not compromised;
- retention/immutability controls remained effective;
- backup software and repositories were not modified;
- restored systems are scanned and monitored in an isolated recovery environment;
- identity, secrets and automation state are rebuilt rather than blindly restored.

12.13 Clean-room recovery

A clean-room recovery environment should use isolated identity, separate administration, controlled data transfer, known-good installation media/images, patched infrastructure, fresh secrets and full telemetry. Restoration order typically follows identity and security services, core network, management, critical applications and user services, but business dependencies must define the actual sequence.

12.14 Ransomware evidence matrix

Question	Memory	Endpoint/disk	Network/cloud	Report outcome	out-
What encrypted files?	Process, handles, strings, active paths	MFT/USN, ransom notes, extension counts, application logs	File-service/storage telemetry	Scope and impact range	
How was it launched?	Parent, command line, token, service context	Service/task/G-PO/remote tool logs	Identity and management API events	Deployment mechanism	
Was code injected?	VAD, malfind, thread start, module lists	EDR process access, extracted binary	Network destination tied to PID	Injection confidence	confi-
Was data exfiltrated?	Archives/config/-sockets/credentials	Staging files and access history	Proxy/DNS/flow/object access	Confirmed/probable/not established	
Were backups attacked?	Admin tools and sessions	Backup agent/config/logs	Vault/API/audit events	Recovery risk	
Which identities are compromised?	Tokens, sessions, credential exposure	Authentication and PAM artifacts	Entra/IAM/federation logs	Reset/revoke scope	
Can systems be re-stored safely?	Active malware/keys/config	Image integrity and persistence	Clean identity/control plane	Recovery confidence	confi-

12.15 Ransomware severity and confidence

Separate **business severity** from **technical confidence**. A single confirmed encryptor on a critical safety system can be severe even before enterprise scope is known. An actor claim of exfiltration may be high business concern but low technical confidence until evidence is collected.

12.16 Ransom-payment evidence

Payment decisions belong to executive, legal, insurer, law-enforcement and sanctions-compliance processes. Forensics supports the decision by establishing affected systems, recovery options, data impact, actor communication provenance and cryptocurrency artifacts. Preserve messages, negotiation records, wallet addresses, transaction IDs and timestamps under access control. Never promise that payment will produce a working decryptor or prevent publication.

Network Forensics Beyond the Memory Image

13.1 Evidence layers

Network investigation combines endpoint sockets with packet, flow, DNS, proxy, firewall, VPN, load balancer, cloud and identity evidence. Each layer answers different questions:

Source	Strength	Limitation
Memory socket	PID/process ownership and run-time state	Point in time; stale structures and incomplete payload
Packet capture	Payload, handshake and precise timing	Limited coverage, encryption and storage volume
Flow logs	Broad direction, bytes, ports and duration	Usually no payload; NAT/identity ambiguity
DNS logs	Domain queries and resolver context	Cache, DoH/DoT, hard-coded IPs and failed-query ambiguity
Proxy logs	URL/user/device and policy context	Only proxied protocols; TLS inspection/privacy constraints
Firewall logs	Allowed/blocked traffic and policy decision	Process identity often absent
VPN/remote access	User/device/session source	Shared accounts, token theft and split tunneling
Cloud load balancer	Requests, clients and backend mapping	Service-specific fields and retention

LOOK FIRST AT

Map every suspicious endpoint to PID, process creation time, user/session, local interface, protocol state and acquisition time. Then determine whether external telemetry confirms the connection.

THEN LOOK AT

Reconstruct sessions, DNS, TLS, NAT, proxy and identity context; calculate scope; identify lateral movement, C2, staging and exfiltration; convert validated behavior into detection logic.

13.2 Connection state interpretation

- **LISTENING**: service exposure; verify bind address, owning process and firewall.
- **ESTABLISHED**: active or recently active session; correlate both directions.
- **SYN_SENT**: attempted outbound connection; may indicate scanning, failed C2 or normal retry.
- **TIME_WAIT/CLOSE_WAIT**: residual protocol state; process ownership may have changed or disappeared.
- **UDP**: no connection state; destination may not be retained in the same way.

13.3 NAT and address translation

A process may connect to a private address while perimeter logs show a public translated address. Record:

- endpoint local tuple;
- host interface and IP at the event time;
- NAT gateway/firewall translation record;
- cloud ENI/NIC and private/public mapping;
- load balancer or proxy intermediate identity.

Do not match solely on port if many clients share NAT. Use time, source address, destination, protocol and translation identifier where available.

13.4 DNS and encrypted DNS

Investigate resolver logs, endpoint DNS telemetry, browser process memory, proxy logs and network captures. DoH/DoT and application-specific resolvers can bypass traditional resolver logs. A DNS query proves an attempt to resolve, not a successful application session. A socket to an IP does not prove which domain caused it without timing and process context.

13.5 TLS analysis

Without session keys, TLS normally reveals metadata rather than plaintext:

- source/destination and timing;
- certificate chain where observable;
- server name indication where not encrypted;
- TLS version, cipher and handshake fingerprint;
- byte counts and session duration.

Memory can contain process-specific configuration, URLs, certificates, bearer tokens or TLS secrets, but extraction must be authorized and tool-specific. Do not assume keys are present or reusable.

13.6 QUIC and HTTP/3

QUIC commonly uses UDP and encrypts most transport metadata. Traditional TCP-state assumptions do not apply. Correlate UDP endpoints, browser/application ownership, DNS, proxy/security telemetry and destination reputation. Blocking UDP/443 during an incident may force protocol fallback but is a containment decision, not a forensic method.

13.7 Packet-capture triage with TShark

These commands operate on preserved captures:

```
tshark -r evidence.pcapng -q -z conv,tcp
tshark -r evidence.pcapng -q -z conv,udp

tshark -r evidence.pcapng -Y 'dns.qry.name' -T fields \
  -e frame.time_epoch -e ip.src -e ip.dst -e dns.qry.name \
  > dns_queries.tsv

tshark -r evidence.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' \
  -T fields -e frame.time_epoch -e ip.src -e ip.dst -e tcp.dstport \
  > tcp_syn.tsv
```

Verify field availability for the capture protocol/version. Keep the original capture immutable and record the TShark version.

13.8 Zeek workflow

On a forensic copy:

```
mkdir zeek-output && cd zeek-output
zeek -r ../evidence.pcapng
sha256sum *.log > zeek-logs.sha256
```

Zeek generates structured connection, DNS, HTTP, TLS and file-related logs depending on traffic. It does not replace packet review and may not decode malformed, encrypted or unsupported protocols.

13.9 Beaconing analysis

Potential beaoning requires a time series. Evaluate:

- inter-arrival interval and jitter;
- destination stability or domain rotation;
- request/response byte pattern;
- process ownership and user activity;
- persistence across hosts;
- expected software update, telemetry and health-check behavior.

Regularity alone is weak evidence because enterprise agents beacon by design. A rare unsigned process connecting at regular intervals to newly observed infrastructure is stronger.

13.10 Lateral-movement patterns

High-value patterns include one source contacting many hosts on SMB/RDP/WinRM/SSH, remote service creation, administrative share access, credential failures followed by success, and remote-management tools outside normal administration windows. Map the network source to process and identity where possible.

13.11 Exfiltration analysis

Estimate:

- first/last outbound event;
- total bytes and session count;
- destination/service/account;
- source process and staged data;
- whether compression or encryption occurred;
- whether transfer completed or failed;
- data classification and accessible repositories.

Report ranges and assumptions. A large outbound session may be backup, replication or software distribution; compare baseline and change records.

13.12 Cloud network evidence

13.12.1 AWS

Correlate EC2 sockets with ENI, VPC Flow Logs, NAT Gateway, Network Firewall, load balancer, Route 53 Resolver and proxy logs. Flow logs may be aggregated and do not contain packet payload. Identify account, region, VPC, subnet, ENI and instance at the event time.

13.12.2 Azure

Correlate VM sockets with NIC, effective NSG, NSG flow data where available, Azure Firewall, Load Balancer/Application Gateway, Network Watcher, DNS and proxy logs. Use resource IDs and Activity Log changes to understand network configuration at the time.

13.13 Network finding template

Endpoint observation	PID/process, local/remote tuple, state, creation time, image hash
External confirmation	Flow/packet/proxy/DNS event identifiers and timestamps
Identity context	User/session/role/device
Destination context	Owner/service/reputation/first seen; avoid reputation-only conclusions
Volume and duration	Bytes, packets, sessions and uncertainty
Interpretation	C2, exfiltration, admin, update, unknown or benign
Next action	Block, capture, scope, extract process, query peers, preserve logs

Reverse Engineering Memory-Derived Artifacts

14.1 Why memory-derived files are different

A process dump, VAD dump or reconstructed module is not necessarily identical to the original disk file. Pages can be missing, modified by relocations, patched, decompressed, privately copied or never present. Import tables and section layout may not be directly executable as a file.

LOOK FIRST AT

Record the source image hash, PID, process create time, VAD start/end, protection, file mapping, extraction plugin/version and export hash.

THEN LOOK AT

Identify format and architecture, compare with disk/reference bytes, map thread entry points, recover configuration, analyze capabilities and create detection content from stable evidence.

14.2 Extraction provenance record

For every export, record:

- case/evidence ID;
- source memory image SHA-256;
- Volatility version and plugin command;
- PID/TID and process image;
- VAD/module/object address and range;
- memory protection and mapped path;
- output filename, size and SHA-256;
- acquisition gaps or plugin warnings.

14.3 Initial identification

```
sha256sum exported.bin
file exported.bin
xxd -l 256 exported.bin
```

```
strings -a -n 6 exported.bin | head
```

On Windows, use PowerShell hashing and a controlled file-identification tool. Do not double-click exports.

14.4 PE analysis

Check:

- DOS/NT headers and architecture;
- section virtual/raw sizes and permissions;
- image base and relocation state;
- imports, delay imports and exports;
- TLS callbacks;
- resources, version information and manifest;
- signature presence and whether the reconstructed bytes can be validated;
- overlay and high-entropy regions;
- entry point versus observed thread start.

An in-memory PE may have resolved imports and relocations but no intact on-disk certificate table. Signature failure on a process dump is therefore not equivalent to an unsigned original file.

14.5 ELF analysis

Check ELF class, machine, type, program headers, dynamic segment, interpreter, needed libraries, symbol/version information, build ID, relocations and deleted/memfd backing. Statically linked Go/Rust binaries and packed ELF files require different expectations from dynamically linked system tools.

14.6 Mach-O analysis

For supported macOS evidence, check architecture, load commands, code-signature structures, entitlements, linked dylibs, hardened-runtime context and whether the extracted bytes represent an intact file. Modern platform restrictions and unsupported acquisition reduce confidence.

14.7 Thread-centric disassembly

Begin disassembly at the exact observed thread start and include surrounding bytes. Determine whether the address is:

- a normal module export or entry point;
- a small trampoline into another region;
- position-independent shellcode;
- a JIT/runtime stub;
- a return-oriented sequence or corrupted pointer.

Map every control transfer to known module/VAD ranges. A jump from anonymous memory into normal APIs is common in both injected code and legitimate runtime stubs.

14.8 Configuration recovery

Configuration may be plaintext, encoded, compressed or decrypted only at runtime. Search the relevant process/VAD for:

- domains, IPs, URLs and user agents;
- campaign/victim IDs;
- mutex, pipe, service and task names;
- exclusion lists and target extensions;
- public keys and cryptographic parameters;
- cloud bucket/container names and API endpoints;
- command templates and sleep intervals.

Validate strings by code reference or runtime relationship. Antivirus engines and browsers can contain malware strings without being compromised.

14.9 FLOSS, capa and Ghidra

Use current verified releases in an isolated analysis VM:

```
floss exported.bin > exported.floss.txt
capa exported.bin > exported.capa.txt
```

Import into Ghidra for structural analysis. Tool findings are leads. Validate capabilities against disassembly and observed runtime evidence.

14.10 Memory-to-disk comparison

When the original file exists:

1. hash and verify the disk file;
2. compare headers, executable sections and signer;
3. distinguish normal loader relocations and copy-on-write pages;
4. identify patched functions or replaced sections;
5. map modifications to thread/network behavior.

A signed disk binary with malicious anonymous code and a thread starting in that code is process injection, not a maliciously signed binary.

14.11 Direct-syscall interpretation

Direct or alternate syscall implementations can occur in security products, sandboxes, compatibility layers and malware. Evaluate:

- module/path ownership;

- whether stubs form a regular generated table;
- protection and allocation type;
- identical presence across related clean processes;
- whether any thread executes the stubs;
- associated process behavior and network activity.

Do not classify a process as malicious because a direct-syscall plugin returns rows.

14.12 YARA engineering from reverse engineering

A strong rule combines multiple stable features and avoids volatile values such as absolute addresses, common compiler stubs or a single domain. Maintain:

- rule ID and version;
- source sample/export hashes;
- analyst and date;
- test corpus and false-positive results;
- scope: file, process VAD, full memory or page cache;
- performance constraints;
- retirement/review date.

14.13 Safe dynamic analysis

Dynamic execution is optional and must occur in an isolated, disposable laboratory with controlled networking, snapshots, full logging and no production credentials. Memory-derived artifacts may not execute because they are incomplete. Never “repair” an artifact into a weaponized sample; the objective is observation and detection.

14.14 Reverse-engineering finding example

FIELD NOTE

A private RX VAD in a signed service contained position-independent x64 code. TID 7712 started 0x180 bytes into the region. The code decoded a configuration containing a domain and named pipe, then invoked networking APIs through addresses resolved from the process. The same domain appeared in proxy logs at the thread creation time. This supports confirmed injected code and probable command-and-control behavior. The signed service binary itself matched the vendor hash.

Kernel and Rootkit Analysis

15.1 Defensive objective

Kernel analysis seeks evidence that code or objects operate outside expected module, driver and callback relationships. It does not attempt to build or conceal rootkits. Modern kernels include many legitimate third-party drivers, security filters, virtualization components and dynamic objects; baseline comparison is essential.

LOOK FIRST AT

Loaded modules, scanned modules, driver objects, driver-to-module mapping, IRP handlers, callbacks, SSDT and orphan kernel threads.

THEN LOOK AT

Timers, unloaded modules, device trees, symbolic links, big pools, extracted drivers, signatures and live-system inventory.

15.2 Module and driver inventory

```
vol.exe -q -f memory.raw -r csv windows.modules.Modules > modules.csv
vol.exe -q -f memory.raw -r csv windows.modscan.ModScan > modscan.csv
vol.exe -q -f memory.raw -r csv windows.driverscan.DriverScan > driverscan.csv
vol.exe -q -f memory.raw -r csv \
    windows.malware.drivermodule.DriverModule > drivermodule.csv
```

Compare by base, size, name and path. Pool-scanned modules absent from the active list can be unloaded modules or false positives. A stronger hidden-driver finding requires an active driver object or execution pointer mapped to that module.

15.3 IRP dispatch analysis

```
vol.exe -q -f memory.raw -r csv windows.driverirp.DriverIrp > driverirp.csv
```

For each driver object, determine whether dispatch routines resolve inside the expected driver or a legitimate framework such as WDF, NDIS or storage components. An address outside all known modules is suspicious only after excluding corrupted driver objects and acquisition artifacts.

15.4 Kernel callbacks

```
vol.exe -q -f memory.raw -r csv windows.callbacks.Callbacks > callbacks.csv
```

Callbacks from antivirus, EDR, filesystem filters, graphics, networking and virtualization drivers are common. Prioritize callback addresses with no module, a user-space-looking address, a hidden module, or a driver with an unexpected path/signature. Code-like constants misread as pointers are scanning false positives.

15.5 SSDT

```
vol.exe -q -f memory.raw -r csv windows.ssdtd.SSDT > ssdt.csv
```

Map each entry to the expected kernel module. PatchGuard and modern Windows defenses make classic SSDT hooking less common, but the check remains useful. Unresolved entries must be validated against symbols, architecture and smearing.

15.6 Orphan kernel threads

```
vol.exe -q -f memory.raw -r csv \  
windows.orphan_kernel_threads.Threads > orphan_threads.csv
```

A kernel thread whose start address is outside loaded modules deserves immediate validation. Check both start and Win32 start addresses, canonical kernel address range, owning PID, module bounds and acquisition corruption. One noncanonical pointer with another address resolving into a known module is often a damaged field, not a rootkit.

15.7 Timers and unloaded modules

```
vol.exe -f memory.raw windows.timers.Timers  
vol.exe -f memory.raw windows.unloadedmodules.UnloadedModules
```

A timer routine should resolve into a known module. Unloaded modules are context, not proof: Windows routinely loads and unloads temporary and device-related drivers. Correlate unload time, driver objects, callbacks and active threads.

15.8 Device tree and symbolic links

```
vol.exe -f memory.raw windows.devicetree.DeviceTree  
vol.exe -f memory.raw windows.symlinkscan.SymlinkScan
```

Unexpected devices and links can reveal stealth drivers or unusual interfaces. Validate against installed hardware, security products, virtualization and acquisition tools.

15.9 Big pools and targeted pool scanning

```
vol.exe -f memory.raw windows.bigpools.BigPools  
vol.exe -f memory.raw windows.poolscanner.PoolScanner
```

These are advanced lead-generation tools. Pool tags are reused, forged and left behind. Use them after a specific hypothesis, not as a primary rootkit verdict.

15.10 Driver extraction and live verification

Use `windows.pedump.PEDump`, `windows.dumpfiles.DumpFiles` or module-specific dumping where supported. On a live reference system, collect service/driver inventory and Authenticode status:

```
Get-CimInstance Win32_SystemDriver |
  Select-Object Name, State, StartMode, PathName |
  Export-Csv live_system_drivers.csv -NoTypeInfoInformation

Get-AuthenticodeSignature 'C:\Windows\System32\drivers\example.sys'
Get-FileHash 'C:\Windows\System32\drivers\example.sys' -Algorithm SHA256
```

15.11 Rootkit decision rule

DECISION RULE

Report a probable kernel rootkit only when at least two independent execution or object relationships converge, such as hidden module + active driver object, unknown driver + callback, unknown driver + IRP dispatch, or orphan kernel thread + executable region not belonging to a legitimate module. A pool-scan difference alone is insufficient.

15.12 Linux rootkit checks

```
python vol.py -f memory.lime linux.lsmmod.Lsmmod
python vol.py -f memory.lime linux.hidden_modules.Hidden_modules
python vol.py -f memory.lime linux.modxview.Modxview
python vol.py -f memory.lime linux.check_modules.Check_modules
python vol.py -f memory.lime linux.check_syscall.Check_syscall
python vol.py -f memory.lime linux.check_idt.Check_idt
python vol.py -f memory.lime linux.check_afinfo.Check_afinfo
python vol.py -f memory.lime linux.check_creds.Check_creds
python vol.py -f memory.lime linux.netfilter.Netfilter
python vol.py -f memory.lime linux.keyboard_notifiers.Keyboard_notifiers
python vol.py -f memory.lime linux.ebpf.EBPF
```

Linux kernel layout is highly build-specific. Exact symbols are mandatory. Namespace, eBPF, tracing and live-patching features can resemble malicious hooks; compare with the distribution baseline and installed agents.

15.13 macOS kernel checks

```
python vol.py -f memory.raw mac.lsmmod.Lsmmod
python vol.py -f memory.raw mac.check_syscall.Check_syscall
python vol.py -f memory.raw mac.check_sysctl.Check_sysctl
python vol.py -f memory.raw mac.check_trap_table.Check_trap_table
python vol.py -f memory.raw mac.kauth_listeners.Kauth_listeners
python vol.py -f memory.raw mac.kauth_scopes.Kauth_scopes
python vol.py -f memory.raw mac.trustedbsd.Trustedbsd
```

Treat results as research-grade on modern macOS because support is not actively maintained.

Windows Memory Forensics: Complete Investigation Runbook

16.1 Scope and operating assumptions

This chapter turns the individual Windows plugins into a controlled investigation sequence. It assumes a legally acquired image, a recorded SHA-256 hash, Volatility 3 version 2.28.0, and symbols resolved through the normal Windows symbol workflow. It also assumes the analyst understands that a live acquisition is not atomic: pages are read over time while the operating system continues to execute.

LOOK FIRST AT

Confirm `windows.info.Info`, acquisition time, architecture, kernel base, DTB, build number and symbol identity. Then create process, thread, network and kernel inventories before selecting anomalies.

THEN LOOK AT

Deep-analyze only candidates supported by two or more observations: parentage, command line, token, network activity, module inconsistency, executable private memory, thread start, persistence, file path, signer or timeline.

16.2 Stage 0: case directory and immutable baseline

```
$Case = 'IR-2026-0042'
$Root = "D:\Cases\$Case"
New-Item -ItemType Directory -Force -Path \
  "$Root\evidence", "$Root\working", "$Root\results", \
  "$Root\exports", "$Root\logs", "$Root\report" | Out-Null

Copy-Item 'E:\Acquisition\host01.raw' "$Root\evidence\host01.raw"
Get-FileHash "$Root\evidence\host01.raw" -Algorithm SHA256 |
  Tee-Object -FilePath "$Root\evidence\host01.sha256.txt"

Copy-Item "$Root\evidence\host01.raw" "$Root\working\host01_working.raw"
vol.exe --version | Out-File "$Root\logs\volatility-version.txt"
vol.exe -h | Out-File "$Root\logs\volatility-plugins.txt"
```

Use access controls so the original is read-only to analysts. Record the source device, operator, acquisition command, start/end times, system time zone, destination medium and any errors. Hash again after transfer.

16.3 Stage 1: validate the image

```
vol.exe -f host01_working.raw windows.info.Info
vol.exe -f host01_working.raw windows.kpcrs.KPCRs
vol.exe -f host01_working.raw windows.statistics.Statistics
```

`windows.info.Info` establishes the kernel and symbol baseline. `windows.kpcrs.KPCRs` can provide processor context. `windows.statistics.Statistics` is useful for understanding layer and symbol conditions, but it is not a malware verdict.

16.3.1 Validation questions

- Does the reported build match inventory, EDR or disk evidence?
- Is the system time plausible relative to acquisition logs?
- Does the processor count match the host or VM?
- Are plugin failures global, or limited to one structure family?
- Do later timestamps appear after the reported system time, indicating smearing?

16.4 Stage 2: process census and cross-view comparison

```
vol.exe -q -f host01_working.raw -r csv windows.pslist.PsList > 01_pslist.csv
vol.exe -q -f host01_working.raw -r csv windows.psscan.PsScan > 02_psscan.csv
vol.exe -q -f host01_working.raw -r csv windows.pstree.PsTree > 03_pstree.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.psxview.PsXView > 04_psxview.csv
vol.exe -q -f host01_working.raw -r csv windows.cmdline.CmdLine > 05_cmdline.csv
vol.exe -q -f host01_working.raw -r csv windows.getsids.GetSIDs > 06_sids.csv
vol.exe -q -f host01_working.raw -r csv windows.privileges.Privs > 07_privileges.csv
vol.exe -q -f host01_working.raw -r csv windows.sessions.Sessions > 08_sessions.csv
vol.exe -q -f host01_working.raw -r csv windows.joblinks.JobLinks > 09_joblinks.csv
```

16.4.1 Interpretation sequence

1. Compare active processes in `PsList` with pool-recovered objects in `PsScan`.
2. Use exit times to separate terminated processes from potentially hidden active processes.
3. Confirm ancestry in `PsTree`; validate unusual parents against service-management behavior and process creation times.
4. Use `PsXView` to compare independent enumerators. Treat one inconsistent column as a lead, not proof.
5. Associate command lines, SIDs, sessions and enabled privileges with each candidate.

FIELD NOTE

A process found by scanning but absent from the active list is commonly terminated. The high-value condition is a process with evidence of continued execution - threads, handles, sockets or current timestamps - while absent from expected active enumerators.

16.5 Stage 3: threads, handles and runtime context

```
vol.exe -q -f host01_working.raw -r csv windows.threads.Threads > 10_threads.csv
vol.exe -q -f host01_working.raw -r csv windows.thrdscan.ThrdScan > 11_thrdscan.csv
vol.exe -q -f host01_working.raw -r csv windows.handles.Handles > 12_handles.csv
vol.exe -q -f host01_working.raw -r csv windows.envvars.Envvars > 13_envvars.csv
vol.exe -q -f host01_working.raw -r csv windows.suspended_threads.SuspendedThreads > 14_suspended.
    csv
vol.exe -q -f host01_working.raw -r csv windows.malware.suspicious_threads.SuspiciousThreads > 15
    _suspicious_threads.csv
```

Look for thread start addresses outside legitimate modules, execution from private VADs, abnormal suspension states, cross-process handles with high access, unusual named sections, pipes, mutant objects and file handles to staging paths. Handles require context: security products, debuggers, accessibility tools and browsers legitimately open high-access process handles.

16.6 Stage 4: module and image integrity

```
vol.exe -q -f host01_working.raw -r csv windows.dlllist.DllList > 20_dlllist.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.lldrmodules.LdrModules > 21_ldrmodules.csv
vol.exe -q -f host01_working.raw -r csv windows.verinfo.VerInfo > 22_verinfo.csv
vol.exe -q -f host01_working.raw -r csv windows.iat.IAT > 23_iat.csv
vol.exe -q -f host01_working.raw -r csv windows.pe_symbols.PESymbols > 24_pe_symbols.csv
```

LdrModules compares VAD-backed images with the process loader lists. A module absent from all lists is more interesting when it is an executable image, belongs to a writable user-controlled path, contains active thread starts or exports code used by the process. Resource mappings, language files, .NET assemblies and the executable's special loader behavior can create expected differences.

16.7 Stage 5: VAD and injection analysis

```
vol.exe -q -f host01_working.raw -r csv windows.vadinfo.VadInfo > 30_vadinfo.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.malfind.Malfind > 31_malfind.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.hollowprocesses.HollowProcesses > 32_hollow.
    csv
vol.exe -q -f host01_working.raw -r csv windows.malware.processghosting.ProcessGhosting > 33
    _ghosting.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.pebmasquerade.PebMasquerade > 34
    _pebmasquerade.csv
vol.exe -q -f host01_working.raw -r csv windows.etwpatch.EtwPatch > 35_etwpatch.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.direct_system_calls.DirectSystemCalls > 36
    _direct_syscalls.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.indirect_system_calls.IndirectSystemCalls >
    37_indirect_syscalls.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.unhooked_system_calls.UnhookedSystemCalls >
    38_unhooked_syscalls.csv
```

16.7.1 VAD triage matrix

Observation	Initial interpretation	Required validation
Private RWX	High-value lead	Thread start, bytes, allocation type, process role, JIT baseline, YARA and extraction
Private RX	Generated or injected code possible	Creator, thread relationship, code structure, repeated presence across clean baseline

Mapped image absent from loader lists	Manual mapping or resource mapping possible	PE header, path, loader-list pattern, active code references
MZ header in anonymous memory	Reflective/manual PE possible	PE structure, imports, relocations, thread start, configuration strings
High-entropy executable VAD	Packed/encrypted content possible	Entropy is not proof; extract, disassemble and correlate
Repeated stubs across related browser processes	Runtime/sandbox mechanism likely	Compare addresses, stub spacing, protection and clean baseline

16.8 Stage 6: persistence and execution history

```
vol.exe -q -f host01_working.raw -r csv windows.svcscan.SvcScan > 40_svcscan.csv
vol.exe -q -f host01_working.raw -r csv windows.svclist.SvcList > 41_svclist.csv
vol.exe -q -f host01_working.raw -r csv windows.malware.svcdiff.SvcDiff > 42_svcdiff.csv
vol.exe -q -f host01_working.raw -r csv windows.registry.scheduled_tasks.ScheduledTasks > 43_tasks.csv
vol.exe -q -f host01_working.raw -r csv windows.amcache.Amcache > 44_amcache.csv
vol.exe -q -f host01_working.raw -r csv windows.shimcachemem.ShimcacheMem > 45_shimcache.csv
vol.exe -q -f host01_working.raw -r csv windows.registry.userassist.UserAssist > 46_userassist.csv
vol.exe -q -f host01_working.raw -r csv windows.registry.hivelist.HiveList > 47_hivelist.csv
```

Memory-resident registry and execution-history artifacts are incomplete by design. Confirm persistence through disk hives, task XML, service registry keys, event logs and EDR. Suspicious characteristics include auto-start entries pointing to user-writable locations, encoded interpreters, network paths, renamed system tools and services whose image path does not match signer or description.

16.9 Stage 7: file objects and targeted extraction

```
vol.exe -q -f host01_working.raw -r csv windows.filescan.FileScan > 50_filesan.csv
vol.exe -o exports -f host01_working.raw windows.dumpfiles.DumpFiles --virtaddr 0xFILE_OBJECT
vol.exe -o exports -f host01_working.raw windows.pslist.PsList --pid 4321 --dump
vol.exe -o exports -f host01_working.raw windows.dlllist.DllList --pid 4321 --dump
vol.exe -o exports -f host01_working.raw windows.malware.malfind.Malfind --pid 4321 --dump
vol.exe -o exports -f host01_working.raw windows.vadinfo.VadInfo --pid 4321 --address 0xADDRESS --dump
vol.exe -o exports -f host01_working.raw windows.memmap.Memmap --pid 4321 --dump
```

Use the exact option names shown by `<plugin> -h`. Hash every export, record the source PID/VAD/object, and preserve the original plugin output. A dump may contain sparse pages, acquisition gaps or reconstruction artifacts and should not be treated as a byte-identical disk executable.

16.10 Stage 8: credential artifact awareness

Windows memory can contain tokens, password-derived material, Kerberos tickets, DPAPI-related material, browser sessions, private keys and secrets held by applications. Plugins such as `windows.cachedump.Cachedump`, `windows.hashdump.Hashdump` and `windows.lsadump.Lsadump` exist, but their use must be explicitly authorized and their output protected as highly sensitive evidence.

CAUTION

Credential extraction changes the risk classification of the case material. Store output encrypted, restrict access, avoid copying secrets into tickets or chat, and initiate credential-reset decisions through the incident commander and identity team. Do not test recovered credentials against production systems without explicit authorization.

16.11 Stage 9: closeout criteria

A Windows memory examination is complete when:

- image and symbol limitations are documented;
- process, network, module, injection, persistence and kernel views were examined;
- anomalies were validated or downgraded using independent evidence;
- relevant exports were hashed and preserved;
- observations were correlated with endpoint, identity, network and cloud telemetry;
- the report distinguishes absence of evidence from evidence of absence.

Linux Memory Forensics

17.1 Why exact symbols matter

Linux kernel structures vary with kernel version, distribution patches, compiler, configuration and build options. A near-match is not a safe match. Volatility's Linux workflow requires an Intermediate Symbol File (ISF) corresponding to the exact kernel banner and relevant debug symbols. The official Linux tutorial and `dwarf2json` documentation provide the supported generation workflow.^[13, 14]

LOOK FIRST AT

Recover the kernel banner, verify acquisition format, locate the exact `vmlinux`/debug package, generate or validate the ISF, then run process and module inventories.

THEN LOOK AT

Inspect process maps, libraries, sockets, files, credentials, kernel modules, syscall tables, IDT, network hooks, eBPF and page cache only after symbol accuracy is established.

17.2 Symbol workflow

```
python3 vol.py -f host01.lime banners.Banners
uname -a
cat /proc/version

# Example ISF generation on a matching analysis system:
./dwarf2json linux --elf /usr/lib/debug/boot/vmlinux-6.x.y > \
  symbols/linux/linux-6.x.y.json

python3 vol.py -f host01.lime -s symbols linux.pslist.PsList
```

The actual debug-symbol path varies by distribution. Keep the uncompressed ISF or a supported compressed representation in the symbol directory. Record the debug package name and hash.

17.3 Process triage

```
python3 vol.py -q -f host01.lime -r csv linux.pslist.PsList > 01_pslist.csv
python3 vol.py -q -f host01.lime -r csv linux.psscan.PsScan > 02_psscan.csv
python3 vol.py -q -f host01.lime -r csv linux.pstree.PsTree > 03_pstree.csv
python3 vol.py -q -f host01.lime -r csv linux.psaux.PsAux > 04_psaux.csv
python3 vol.py -q -f host01.lime -r csv linux.pidhashtable.PIDHashTable > 05_pidhashtable.csv
python3 vol.py -q -f host01.lime -r csv linux.kthreads.Kthreads > 06_kthreads.csv
python3 vol.py -q -f host01.lime -r csv linux.envvars.Envvars > 07_envvars.csv
python3 vol.py -q -f host01.lime -r csv linux.capabilities.Capabilities > 08_capabilities.csv
```

Investigate processes with deleted executables, unexpected namespaces, unusual parentage, odd command-line arguments, capabilities inconsistent with role, processes running from writable mounts, hidden task-list discrepancies or container-runtime context that does not match orchestration records.

17.4 Process memory and injected code

```
python3 vol.py -q -f host01.lime -r csv linux.proc.Maps > 10_maps.csv
python3 vol.py -q -f host01.lime -r csv linux.library_list.LibraryList > 11_libraries.csv
python3 vol.py -q -f host01.lime -r csv linux.malware.malfind.Malfind > 12_malfind.csv
python3 vol.py -f host01.lime linux.vmayarascan.VmaYaraScan \
    --pid 4321 --yara-file rules.yar
python3 vol.py -f host01.lime linux.vmaregexscan.VmaRegExScan \
    --pid 4321 --pattern 'example[.]invalid'
```

Linux anonymous executable mappings can be legitimate for JIT engines, language runtimes, seccomp trampolines and security agents. Stronger findings combine executable anonymous memory, deleted backing files, active instruction pointers, network behavior and extracted configuration.

17.5 Files, mounts and page cache

```
python3 vol.py -q -f host01.lime -r csv linux.lsof.Lsof > 20_lsof.csv
python3 vol.py -q -f host01.lime -r csv linux.mountinfo.MountInfo > 21_mounts.csv
python3 vol.py -q -f host01.lime -r csv linux.pagecache.Files > 22_pagecache_files.csv
python3 vol.py -f host01.lime linux.pagecache.InodePages --inode 0xINODE
python3 vol.py -o exports -f host01.lime linux.pagecache.RecoverFs
```

Page-cache recovery can reconstruct evidence that no longer exists in directory listings, but recovered bytes may be partial. Preserve inode, mount and path context and compare with filesystem metadata.

17.6 Network analysis

```
python3 vol.py -q -f host01.lime -r csv linux.ip.Addr > 30_ip_addr.csv
python3 vol.py -q -f host01.lime -r csv linux.ip.Link > 31_ip_link.csv
python3 vol.py -q -f host01.lime -r csv linux.sockstat.Sockstat > 32_sockstat.csv
python3 vol.py -q -f host01.lime -r csv linux.sockscan.Sockscan > 33_sockscan.csv
```

Map sockets to tasks, namespaces and containers. Distinguish a host-network process from a container-network process. Correlate remote endpoints with VPC/VNet flow logs, load-balancer logs, DNS and egress proxies.

17.7 Shell and interactive-session artifacts

```
python3 vol.py -f host01.lime linux.bash.Bash
python3 vol.py -f host01.lime linux.tty_check.tty_check
python3 vol.py -f host01.lime linux.keyboard_notifiers.Keyboard_notifiers
```

Bash history recovered from memory is opportunistic. Commands may be absent, duplicated or unrelated to the current incident. Shells can disable history, delay writes, use alternative shells or execute through automation. Treat recovered commands as leads and correlate with auditd, journald, sudo logs, terminal recordings, EDR and cloud session logs.

17.8 Kernel and rootkit workflow

```
python3 vol.py -q -f host01.lime -r csv linux.lsmmod.Lsmmod > 40_lsmmod.csv
python3 vol.py -q -f host01.lime -r csv linux.hidden_modules.Hidden_modules > 41_hidden_modules.csv
python3 vol.py -q -f host01.lime -r csv linux.modxview.Modxview > 42_modxview.csv
python3 vol.py -q -f host01.lime -r csv linux.check_modules.Check_modules > 43_check_modules.csv
python3 vol.py -q -f host01.lime -r csv linux.check_syscall.Check_syscall > 44_syscalls.csv
python3 vol.py -q -f host01.lime -r csv linux.check_idt.Check_idt > 45_idt.csv
python3 vol.py -q -f host01.lime -r csv linux.check_afinfo.Check_afinfo > 46_afinfo.csv
python3 vol.py -q -f host01.lime -r csv linux.check_creds.Check_creds > 47_creds.csv
python3 vol.py -q -f host01.lime -r csv linux.netfilter.Netfilter > 48_netfilter.csv
python3 vol.py -q -f host01.lime -r csv linux.ebpf.EBPF > 49_ebpf.csv
python3 vol.py -q -f host01.lime -r csv linux.tracing.ftrace.CheckFtrace > 50_ftrace.csv
python3 vol.py -q -f host01.lime -r csv linux.tracing.perf_events.PerfEvents > 51_perf.csv
python3 vol.py -q -f host01.lime -r csv linux.tracing.tracepoints.CheckTracepoints > 52_tracepoints.csv
```

Modern Linux observability and security products legitimately use eBPF, tracepoints, ftrace, kprobes and netfilter. A rootkit conclusion requires ownership and provenance analysis: program name, map, attach point, loaded module, package, service, signer where applicable, deployment manifest and baseline.

17.9 Container and Kubernetes considerations

Memory belongs to the node, not to a single logical container. PID namespaces can cause the same task to have different host and container PIDs. Overlay filesystems can produce paths that differ from the image layer. Investigate:

- container runtime processes and shims;
- namespace membership and cgroups;
- container image digest and deployment manifest;
- hostPath mounts and privileged containers;
- service-account tokens and cloud workload identity;
- node sockets, eBPF programs and CNI components;
- EKS/AKS control-plane audit logs and admission events.

AWS's EKS incident-response guidance explicitly recommends volatile capture of worker-node memory, process/network state and container runtime state when authorized and operationally feasible.^[21]

17.10 Linux acquisition limitations

AVML can be blocked by kernel lockdown; LiME requires loading a kernel module and therefore changes kernel state; kdump uses a crash kernel and may filter pages depending on configuration. The default forensic question should be: which pages and physical ranges were omitted, transformed or compressed, and can Volatility reconstruct the expected address space?^[10, 11, 22]

macOS Memory Forensics: Realistic 2026 Scope

18.1 Support boundary

Volatility's official macOS tutorial states that macOS support is no longer actively maintained. That warning must govern conclusions. Plugin presence does not guarantee reliable analysis of current Apple Silicon, current kernel structures or modern memory-protection mechanisms.[6]

LOOK FIRST AT

Identify Intel versus Apple Silicon, exact macOS and Darwin kernel version, acquisition method, whether the image is a full physical capture, and whether a matching symbol table can be produced.

THEN LOOK AT

Use Volatility only when the image and symbols are demonstrably supported. Otherwise prioritize unified logs, Endpoint Security telemetry, disk snapshots, process samples, crash reports, network logs and MDM/EDR evidence.

18.2 Acquisition reality

There is no universal, current, free, low-impact full-physical-memory acquisition method for modern Apple Silicon comparable to a routinely supported Windows or Linux workflow. Legacy `osxpmem` workflows apply to controlled Intel-era systems and should not be represented as a modern universal solution. In virtualized research environments, hypervisor-level capture may be possible depending on platform and format.

18.3 Symbol and plugin workflow

When a supported image and matching ISF are available:

```
python3 vol.py -f mac.raw mac.pslist.PsList
python3 vol.py -f mac.raw mac.pstree.PsTree
python3 vol.py -f mac.raw mac.psaux.PsAux
python3 vol.py -f mac.raw mac.proc_maps.Maps
python3 vol.py -f mac.raw mac.malfind.Malfind
python3 vol.py -f mac.raw mac.lsof.Lsof
python3 vol.py -f mac.raw mac.netstat.Netstat
python3 vol.py -f mac.raw mac.ifconfig.Ifconfig
python3 vol.py -f mac.raw mac.list_files.List_Files
python3 vol.py -f mac.raw mac.mount.Mount
```

18.4 Kernel-oriented checks

```
python3 vol.py -f mac.raw mac.lsmmod.Lsmmod
python3 vol.py -f mac.raw mac.check_syscall.Check_syscall
python3 vol.py -f mac.raw mac.check_sysctl.Check_sysctl
python3 vol.py -f mac.raw mac.check_trap_table.Check_trap_table
python3 vol.py -f mac.raw mac.kauth_listeners.Kauth_listeners
python3 vol.py -f mac.raw mac.kauth_scopes.Kauth_scopes
python3 vol.py -f mac.raw mac.socket_filters.Socket_filters
python3 vol.py -f mac.raw mac.timers.Timers
python3 vol.py -f mac.raw mac.trustedbsd.Trustedbsd
python3 vol.py -f mac.raw mac.vfsevents.VFSevents
```

Results require a known-good baseline for the same OS build and hardware. System extensions, Endpoint Security clients, network extensions, virtualization and Apple security mechanisms can resemble monitoring hooks.

18.5 Professional alternatives

For modern macOS investigations, preserve and correlate:

- unified logs and log archives;
- Endpoint Security/EDR process, file and network events;
- APFS snapshots and disk images;
- launch agents, launch daemons and login items;
- system extensions and network extensions;
- quarantine and download metadata;
- browser and application artifacts;
- process samples, crash reports and diagnostic collections;
- MDM state, FileVault state and identity-provider logs.

DECISION RULE

Do not publish a clean-memory conclusion for a modern macOS host when acquisition or symbol support is uncertain. State that full physical-memory examination was unavailable or not validated, and report conclusions from the evidence planes actually collected.

Endpoint Forensics Integration

19.1 Memory is not a standalone investigation

Memory analysis is strongest when combined with disk, logs and telemetry. Endpoint evidence can establish persistence, original file bytes, prefetch/execution traces, service installation, file-system chronology, EDR detections and user activity that no longer exists in RAM.

LOOK FIRST AT

Correlate each high-priority PID or module with its on-disk path, signer, hash, file timestamps, execution artifacts, service/task registration, EDR process graph and user identity.

THEN LOOK AT

Build a cross-source timeline, recover deleted/staged files, reverse engineer extracted code, and identify scope across peer hosts through hashes, paths, command-line fragments, certificates and behavioral detections.

19.2 Windows endpoint evidence

High-value sources include:

- Windows event logs: Security, System, PowerShell, Task Scheduler, WMI and Defender;
- Microsoft-Windows-Sysmon/Operational when deployed;
- Prefetch, Amcache, Shimcache and SRUM;
- NTFS MFT, USN Journal and transaction logs;
- registry hives, services, scheduled tasks and WMI subscriptions;
- Defender/EDR quarantine and alert records;
- browser, RDP, VPN and remote-management artifacts;
- volume-shadow-copy and backup events.

19.3 Linux endpoint evidence

Prioritize journald, auditd, authentication logs, package manager history, shell and sudo logs, cron/sys-temd units, SSH authorized keys, container runtime records, filesystem journals, deleted files, cloud-init, agent logs and EDR telemetry.

19.4 macOS endpoint evidence

Prioritize unified logs, FSEvents, APFS snapshots, launchd configuration, system/network extensions, quarantine metadata, TCC, browser artifacts, Endpoint Security telemetry, MDM records and identity logs.

19.5 Static and reverse-engineering workflow

For each extracted executable or VAD:

1. Calculate SHA-256 and record provenance.
2. Identify file type and architecture.
3. Inspect PE/ELF/Mach-O headers and import/export structure.
4. Run defensive capability analysis such as capa.
5. Extract obfuscated strings using FLOSS where applicable.
6. Disassemble in Ghidra or another controlled reverse-engineering environment.
7. Compare code with VAD bytes and thread entry point.
8. Derive YARA only from stable, distinctive features.
9. Never execute the sample outside an isolated, authorized lab.

19.6 Signer and hash verification

```
Get-AuthenticodeSignature 'C:\Path\sample.exe' | Format-List
Get-FileHash 'C:\Path\sample.exe' -Algorithm SHA256

sha256sum sample.bin
file sample.bin
readelf -h sample.bin
```

A valid signature proves that the file matches content signed by the certificate at signing/validation time; it does not prove that the signer is trustworthy for the current context or that a signed process was not injected.

AWS Security Incident Response and Forensics

20.1 Cloud evidence model

AWS investigations combine guest evidence, service control-plane logs, identity events, network telemetry, storage snapshots and provider records. The shared-responsibility model changes collection: the responder controls guest operating systems and customer configuration, while AWS controls underlying infrastructure and many provider logs.

AWS Security Incident Response documentation updated in 2026 describes EC2 Triage through Systems Manager for collecting volatile host state such as processes, connections and logged-in users. It is not equivalent to a full physical-memory image. AWS also documents automated forensic orchestration patterns for EC2 and EKS.[12, 23]

LOOK FIRST AT

Preserve CloudTrail, identity context, affected account/region, instance metadata, security groups, ENIs, EBS volume identifiers, AMI, instance profile, tags, VPC Flow Logs, GuardDuty/Security Hub findings and relevant application logs.

THEN LOOK AT

Isolate without terminating, snapshot disks, collect guest volatile state or memory where authorized, export logs to a dedicated forensic account, analyze role sessions and API chains, and scope peer resources through account-wide queries.

20.2 Preparation architecture

A mature AWS program maintains:

- a dedicated security/forensic account;
- cross-account responder roles with least privilege;
- organization-wide CloudTrail and protected log storage;
- centralized GuardDuty, Security Hub and Config;
- immutable or retention-controlled evidence buckets;
- prebuilt isolation security groups;
- SSM access for approved triage;

- tested EBS snapshot sharing and KMS permissions;
- automated tagging and chain-of-custody records;
- regional playbooks because resources and evidence are region-scoped.

20.3 Immediate containment without destroying evidence

Create or pre-stage an isolation security group that allows only required forensic management paths. Then attach it to the affected ENI:

```
aws ec2 modify-network-interface-attribute \
  --network-interface-id eni-0123456789abcdef0 \
  --groups sg-0forensicisolation
```

Changing security groups is a control-plane action and should be recorded in CloudTrail. Verify that dependent interfaces, load balancers, EFS mounts, SSM connectivity and clustered applications are understood before isolation. Do not terminate the instance before volatile collection.

20.4 EBS snapshot acquisition

For all EBS volumes attached to an instance:

```
aws ec2 create-snapshots \
  --instance-specification InstanceId=i-0123456789abcdef0 \
  --description 'IR-2026-0042 pre-containment evidence' \
  --copy-tags-from-source volume
```

For a single volume:

```
aws ec2 create-snapshot \
  --volume-id vol-0123456789abcdef0 \
  --description 'IR-2026-0042 root volume evidence'
```

Snapshots are crash-consistent unless the guest and application are quiesced. Record snapshot IDs, source volume IDs, account, region, KMS key, start/completion times and API caller. Copy or share evidence into the forensic account according to encryption and legal constraints.

20.5 CloudTrail preservation and querying

```
aws cloudtrail lookup-events \
  --start-time '2026-07-27T00:00:00Z' \
  --end-time '2026-07-28T00:00:00Z' \
  --lookup-attributes \
    AttributeKey=ResourceName,AttributeValue=i-0123456789abcdef0 \
  --output json > cloudtrail_instance.json
```

`lookup-events` searches event history with limitations; organization trails, CloudTrail Lake or the centralized log store are required for broader and longer-range analysis. Preserve raw events, event IDs, user identity, session issuer, source IP, user agent, request parameters, response elements and related resources.

20.6 Identity-chain analysis

Reconstruct:

1. initial principal or access key;
2. MFA and federation context;
3. `AssumeRole` chain and session name;
4. source identity and external ID where present;
5. policy changes and newly attached permissions;
6. access-key creation, console login and credential-report changes;
7. API calls to S3, KMS, Secrets Manager, EC2, IAM, Lambda and CloudTrail;
8. data-plane access logs where enabled.

The same public IP can represent NAT, VPN, proxy or cloud infrastructure. Identity-chain and behavior analysis is more reliable than IP reputation alone.

20.7 Guest memory and EC2 triage

A full guest memory acquisition uses an OS-specific collector inside the instance or an approved automated workflow. This alters guest state. AWS EC2 Triage through SSM gathers volatile process/network/system information but should not be described as full physical RAM. Preserve command execution IDs and SSM output as evidence.^[12]

For EKS worker nodes, capture node memory and runtime state only after understanding workload disruption, namespace boundaries and managed-node replacement behavior.^[21]

20.8 AWS ransomware investigation

Ransomware in AWS may involve:

- guest encryption inside EC2;
- deletion or encryption of EBS snapshots and backups;
- S3 object deletion, overwrite or server-side encryption changes;
- KMS key-policy changes or scheduled key deletion;
- IAM credential theft and privilege escalation;
- disabling CloudTrail, GuardDuty or logging destinations;
- exfiltration through S3, snapshots, databases or application APIs;
- compromise of backup accounts or cross-account trust.

DECISION RULE

Do not assume that encrypted EC2 files imply an AWS control-plane compromise. Conversely, a clean guest memory image does not exclude stolen IAM credentials, malicious API activity or snapshot deletion. Scope both planes independently and then correlate them.

20.9 AWS evidence checklist

Evidence plane	Examples
----------------	----------

Identity	IAM users/roles, federation, SSO, access keys, credential reports, role sessions
Control plane	CloudTrail management events, Organizations, Config, KMS, account settings
Network	VPC Flow Logs, Traffic Mirroring, load balancer, Route 53 resolver, firewall and proxy logs
Compute	EC2 triage, RAM where authorized, EBS snapshots, AMI, user data, SSM history
Storage	S3 data events, object versions, access logs, inventory, replication and Object Lock state
Detection	GuardDuty, Security Hub, Inspector, Detective and partner telemetry
Application	CloudWatch logs, database audit logs, API Gateway, Lambda and container logs

Azure Security Incident Response and Forensics

21.1 Azure evidence model

Azure incident response combines guest evidence, Azure Resource Manager activity, Microsoft Entra identity logs, resource diagnostic logs, network telemetry, VM/disk snapshots and Microsoft security products. Microsoft's July 2026 incident-response overview identifies Entra logs, Azure Activity Logs, NSG flow data, diagnostic logs and VM snapshots as key sources.[24, 25]

LOOK FIRST AT

Identify tenant, subscription, resource group, region, resource IDs, managed identities, role assignments, VM/NIC/disk identifiers, Activity Log events, Entra sign-ins/audits, Defender for Cloud alerts and relevant diagnostic settings.

THEN LOOK AT

Contain through network and identity controls, snapshot disks, preserve cloud logs, acquire guest volatile evidence where authorized, investigate role assignments and application identities, and export evidence into a dedicated subscription/storage account.

21.2 Isolation through NSG reassignment

```
az network nic update \
  --ids /subscriptions/SUB/resourceGroups/RG/providers/\
Microsoft.Network/networkInterfaces/NIC01 \
  --network-security-group /subscriptions/SUB/resourceGroups/IR-RG/\
providers/Microsoft.Network/networkSecurityGroups/nsg-forensic-isolation
```

An NSG change is visible in the Activity Log. Validate effective rules, load balancer health probes, Bastion/management paths, service endpoints and private endpoints. Do not deallocate the VM before volatile collection unless the containment decision outweighs evidence loss.

21.3 Managed-disk snapshot acquisition

```
DISK_ID=$(az vm show -g prod-rg -n app01 \
  --query 'storageProfile.osDisk.managedDisk.id' -o tsv)

az snapshot create \
  --resource-group ir-evidence-rg \
```

```
--name IR-2026-0042-app01-osdisk \
--source "$DISK_ID" \
--tags Case=IR-2026-0042 EvidenceType=DiskSnapshot
```

A snapshot of a running disk is generally crash-consistent. Record source resource ID, snapshot ID, region, encryption configuration, operator, timestamp and Activity Log event. Azure's chain-of-custody reference architecture highlights that platform audit logs can document who created a snapshot and when.^[26]

21.4 Activity Log collection

```
az monitor activity-log list \
--start-time 2026-07-27T00:00:00Z \
--end-time 2026-07-28T00:00:00Z \
--resource-id /subscriptions/SUB/resourceGroups/prod-rg/\
providers/Microsoft.Compute/virtualMachines/app01 \
--output json > activity-app01.json
```

The Activity Log covers subscription-level control-plane operations, not every data-plane access. Export relevant logs to Log Analytics, Event Hubs or protected storage before normal retention expires.^[27]

21.5 Entra identity investigation

Examine:

- interactive and non-interactive sign-ins;
- service-principal and managed-identity sign-ins;
- conditional-access results and device context;
- risky users and risky sign-ins;
- application registrations, credentials and consent grants;
- directory-role and Azure RBAC assignments;
- federated identity credentials;
- token replay indicators and anomalous client applications;
- mailbox and Microsoft 365 audit logs when scope crosses services.

Residential proxies and cloud egress can reduce IP-based value. Behavioral sequences, application IDs, token claims, consent changes and resource access are stronger.

21.6 KQL examples for defensive hunting

The following queries are templates; table availability depends on enabled connectors and workspace schema.

```
AzureActivity
| where TimeGenerated between (datetime(2026-07-27) .. datetime(2026-07-28))
| where ResourceId =~ '/subscriptions/SUB/resourceGroups/prod-rg/providers/Microsoft.Compute/
virtualMachines/app01'
| project TimeGenerated, Caller, OperationNameValue, ActivityStatusValue,
CallerIpAddress, CorrelationId, Properties
| order by TimeGenerated asc
```

```
SigninLogs
| where TimeGenerated > ago(7d)
| summarize Attempts=count(), Countries=dcount(LocationDetails.countryOrRegion),
              IPs=dcount(IPAddress) by UserPrincipalName, AppDisplayName
| where Attempts > 20 or Countries > 3 or IPs > 10
| order by Attempts desc
```

Do not copy a query into a report without validating the actual table, field semantics, licensing and retention in the affected tenant.

21.7 Guest memory and VM triage

Azure supports disk snapshots and recommends the guest operating system’s native memory-dump mechanism where needed. A full memory image generally requires an in-guest collector or crash dump; the hypervisor’s host memory is not customer-accessible.[\[25\]](#)

If using `az vm run-command invoke`, document that it executes inside the guest and modifies state. Prefer a pre-installed, signed collection package and capture Run Command metadata/output.

21.8 Azure ransomware investigation

Investigate both guest and control planes:

- file encryption and destructive commands in the VM;
- disk/snapshot/backup deletion;
- Recovery Services vault and immutable-vault configuration changes;
- Key Vault access-policy or RBAC changes;
- storage account key regeneration and object deletion;
- new role assignments and managed-identity abuse;
- diagnostic-setting deletion or log-routing changes;
- exfiltration through Storage, databases, Graph, mail or application APIs;
- compromise of Entra applications and OAuth consent.

21.9 Microsoft Sentinel investigation workflow

Use Sentinel incidents and hunting to correlate Entra, Defender XDR, Azure Activity, VM security events, network logs and threat intelligence. Preserve original records and alert IDs. Analytics and entity mapping help triage, but the underlying raw events remain the evidentiary basis.[\[28, 29\]](#)

On-Premises, Cloud and Hybrid Investigations

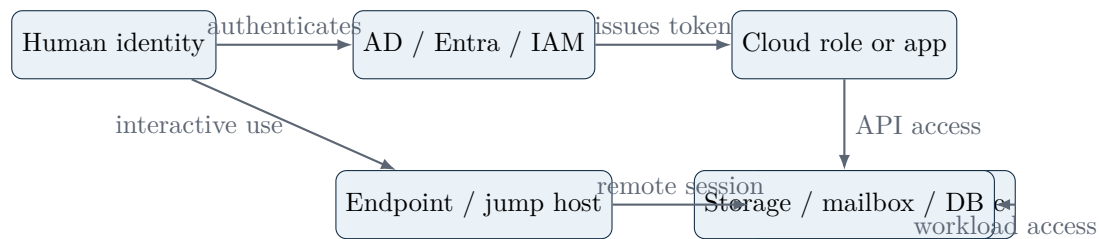
22.1 Core differences

Dimension	On-premises	Cloud	Hybrid
Authority	Physical and administrative control may be direct	Shared responsibility and service/API boundaries	Multiple owners, providers and identity planes
Volatile acquisition	Hardware/OS/hypervisor methods	Usually in-guest; provider host unavailable	Method differs by workload location
Containment	VLAN, switch, EDR, host firewall, power decision	SG/NSG, IAM, API controls, snapshots, service isolation	Must avoid breaking trust paths and evidence sources
Identity	AD/local accounts, VPN, PAM	IAM/Entra, roles, tokens, applications, federation	AD, Entra/IAM, sync, federation and service accounts
Network	Packet capture, firewall, proxy, DNS	Flow logs, load balancers, cloud firewalls, service logs	Cross-boundary egress and private connectivity
Storage evidence	Physical/logical disks, SAN snapshots	EBS/managed-disk/object versions/snapshots	Duplicate and inconsistent retention models
Timeline	Host clocks and central logs	Provider event time, ingestion time and service-specific time	Normalize multiple clocks and control/data planes
Eradication	Reimage, patch, credential reset	Rebuild from image/IaC, rotate roles/keys/tokens	Coordinate both; remove persistence across trust boundary

22.2 The hybrid evidence graph

A hybrid incident should be represented as an evidence graph rather than a host list. Nodes include human identities, service principals, roles, endpoints, VMs, containers, applications, mailboxes, storage, keys and network segments. Edges include sign-ins, role assumption, process creation, socket connection,

API call, object access, remote session and trust relationship.



22.3 Containment order in hybrid ransomware

1. Stop destructive automation and revoke actively abused identities.
2. Preserve cloud audit and identity logs before retention or attacker deletion.
3. Isolate affected endpoints/VMs without terminating volatile workloads when safe.
4. Protect backups, vaults, snapshots, KMS/Key Vault and administrative accounts.
5. Block known lateral paths while preserving necessary forensic management.
6. Capture volatile evidence and disk snapshots.
7. Scope trust relationships, synchronization, federation, remote management and service accounts.
8. Rebuild from trusted sources; do not restore compromised identity or automation state.

Timeline Construction and Evidence Correlation

23.1 Time fields are not equivalent

A professional timeline distinguishes:

- event time reported by the source;
- system clock and time-zone context;
- acquisition start/end;
- log ingestion time;
- provider processing time;
- file-system create/modify/change/access times;
- process create/exit and socket create times;
- analyst collection/export time.

Normalize to UTC while preserving the original field and source. Never overwrite original timestamps during export.

23.2 Volatility timeliner

```
vol.exe -q -f memory.raw -r csv timeliner.Timeliner > volatility_timeline.csv
```

`timeliner.Timeliner` aggregates timestamp-producing plugins, but completeness depends on image, OS and available plugin data. Treat it as one source, not the entire incident chronology.

23.3 Correlation keys

Use stable identifiers wherever possible:

Evidence type	Useful correlation keys
Process	PID plus create time, image path, hash, parent PID/create time, logon/session ID

Network	protocol, local/remote tuple, PID/create time, interface, flow identifier
Windows	logon ID, process GUID, service name, task path, registry key, file ID
Linux	PID namespace, cgroup, inode, mount ID, container ID, systemd unit
AWS	account, region, event ID, principal ARN, session issuer, resource ARN, request ID
Azure	tenant, subscription, resource ID, correlation ID, caller/object ID, application ID
File	path, file ID/inode, volume, hash, signer, size and timestamp set

23.4 Memory-smearing controls

Use acquisition start and end as a window. If an object exit time occurs after the kernel's reported time, do not force a single instant. Compare multiple structures and report that the image represents pages captured over a range.

23.5 Hypothesis table

Maintain a live table:

Hypothesis	Supporting evidence	evi-	Contradicting evidence	evi-	Next test
Process 4321 injected	Private RX thread start inside VAD;		Signed image; no network		Dump VAD, YARA, module baseline
IAM role compromised	New role session; anomalous S3 calls		Source IP is corporate VPN		Trace federation and device identity
Kernel rootkit active	Module only in mod-scan		Module appears unloaded; no callback		Map driver objects and IRP handlers

IOC Extraction, Threat Hunting and Detection Engineering

24.1 IOC hierarchy

Indicators have different durability:

1. hashes and exact IPs are precise but often short-lived;
2. paths, mutexes, service names and certificates can survive variants;
3. command-line fragments, API sequences and parent-child behavior are more durable;
4. identity and cloud permission changes can be campaign-level behavior;
5. TTP-based detections are durable but generate more false positives.

24.2 Extraction workflow

For every confirmed or probable finding, extract:

- SHA-256 of disk and memory-derived objects;
- path, filename, signer and certificate thumbprint;
- process/parent names and command lines;
- service, task, registry and WMI identifiers;
- IP, port, protocol, domain, URL and user agent;
- named pipes, mutants, sections and unusual handles;
- driver/module names and callback types;
- cloud principal, role/session, application ID, resource ID and API operation;
- first/last observed times and evidence source.

24.3 YARA from memory findings

YARA rules should use stable byte sequences, strings and structural constraints. Avoid rules based only on packer markers or common API names. Test against clean operating-system, browser, runtime and security-product corpora.

```
rule Lab_Example_ConfigMarker_2026 {
  meta:
    author = "OxAllow"
    purpose = "Defensive lab example"
  strings:
    $marker = "LAB-C2-CONFIG-v1" ascii wide
    $uri = "/lab-safe/checkin" ascii
  condition:
    uint16(0) == 0x5A4D and all of them
}
```

This rule is intentionally synthetic. Production rules require provenance, versioning, test results and false-positive documentation.

24.4 Sigma and log mapping

Memory findings can become log detections. Examples:

- suspicious parent/child and command line -> process creation analytics;
- service or task persistence -> Windows event/Sysmon/Security log rules;
- direct cloud role abuse -> CloudTrail/Azure Activity/Entra analytics;
- unusual remote endpoint -> proxy, firewall and flow-log detection;
- driver load -> kernel/Code Integrity/EDR rules.

Sigma describes log-oriented detection logic and does not scan memory. YARA scans byte-oriented content and does not replace behavioral detection.

24.5 MITRE ATT&CK mapping

Map only behaviors supported by evidence. A process name is not a technique. Examples:

- confirmed process injection -> T1055 and appropriate sub-technique;
- malicious service installation -> T1543.003;
- scheduled task persistence -> T1053.005;
- credential dumping -> T1003, with the observed method;
- cloud account or application abuse -> relevant valid-account and cloud-account techniques;
- data encrypted for impact -> T1486;
- inhibit system recovery -> T1490;
- exfiltration -> map the observed channel, not merely ransomware family reputation.

24.6 Enterprise hunting package

A finding should produce a package containing:

1. normalized IOC CSV/JSON;
2. YARA rules with test corpus results;
3. Sigma or platform-native queries;

4. EDR hunt for process/module/path/hash behavior;
5. cloud queries for principals, APIs and resources;
6. time range and environment assumptions;
7. known false positives and rollback plan.

Verified Incident Lessons

25.1 Use of public incidents

The cases below use public facts from government, regulatory or vendor primary sources. They do not claim access to private forensic images. Each lesson separates published facts from a hypothetical evidence plan that a responder could use in a similar authorized incident.

25.2 Colonial Pipeline ransomware response

The U.S. Department of Justice reported the seizure of cryptocurrency associated with a ransom payment following the May 2021 DarkSide incident. Public reporting established major operational disruption and a ransomware/extortion context.[18]

25.2.1 Defensive lessons

- Business shutdown decisions can exceed the direct technical blast radius.
- Identity, VPN and remote-access evidence must be preserved with endpoint evidence.
- Payment artifacts, wallet information and communications become separate evidentiary streams.
- Recovery requires validation of identity, segmentation and operational technology dependencies, not only decryption.

25.2.2 Comparable investigation plan

Capture affected access gateways, administrative endpoints and encryption hosts; preserve VPN/authentication logs, process/network memory artifacts, domain-controller evidence, EDR timelines, backup control-plane records and exfiltration telemetry. Do not infer a specific initial-access technique unless supported by the evidence.

25.3 Change Healthcare disruption

UnitedHealth Group disclosed that it identified unauthorized access in the Change Healthcare environment and isolated affected systems. The event produced broad healthcare operational disruption and later public disclosures regarding data impact.[19]

25.3.1 Defensive lessons

- A critical third-party platform can create sector-wide dependency impact.
- Isolation can be necessary before full scope is known, but it changes volatile evidence.
- Identity, remote access, payment systems, claims workflows and data-exfiltration evidence must be investigated together.
- Regulatory, privacy and patient-care consequences shape evidence priorities.

25.4 ESXiArgs campaign

CISA published recovery guidance for ESXiArgs incidents affecting VMware ESXi systems.[20]

25.4.1 Defensive lessons

Hypervisor ransomware requires evidence from the management plane, ESXi logs, datastore metadata, virtual disks and network/security infrastructure. Guest memory may be unavailable after encryption or shutdown. Maintain offline management-plane logs and tested VM recovery procedures.

25.5 MGM Resorts and identity-centric intrusion

MGM Resorts disclosed a cybersecurity issue affecting U.S. systems, operational disruption and an estimated financial impact. CISA separately documented Scattered Spider tactics including sophisticated social engineering and identity-focused intrusion methods; public evidence should not be overextended beyond what each source states.[30, 31]

25.5.1 Defensive lessons

- Help-desk and identity workflows are part of the attack surface.
- Endpoint memory may show remote-management sessions and tools, while decisive evidence may reside in identity, ticketing, telephony and MFA logs.
- Restoring endpoints without invalidating sessions, tokens and compromised administrative workflows is inadequate.

25.6 Capital One cloud breach

The U.S. Department of Justice described unauthorized access to misconfigured AWS environments and theft of data from multiple entities, including Capital One.[32]

25.6.1 Defensive lessons

- Cloud resource configuration and identity permissions are evidence, not background configuration.
- Guest memory can reveal tools and credentials, but CloudTrail, storage access and role activity define cloud scope.
- Investigators must reconstruct the control-plane/data-plane chain rather than treating the event as a single compromised server.

25.7 Midnight Blizzard and OAuth application abuse

Microsoft reported that Midnight Blizzard used password spraying against a legacy non-production tenant account, abused OAuth applications and accessed corporate email. Microsoft emphasized EWS and audit-log investigation and noted that fast-changing residential proxy IPs reduce the value of simple IOC matching.[33]

25.7.1 Defensive lessons

- Cloud identity persistence can survive password reset through applications, service principals and consent grants.
- Hunt for authorization changes and API behavior, not just IP addresses.
- Memory forensics of an endpoint cannot prove a SaaS tenant is clean.

25.8 Storm-0558 and cloud-token assurance

The U.S. Cyber Safety Review Board's review of the 2023 Microsoft Exchange Online intrusion focused attention on cloud identity, token signing, logging and provider/customer visibility.[34]

25.8.1 Defensive lessons

High-impact cloud investigations may depend on provider-side token and key evidence unavailable to customers. Response plans must include provider escalation, log entitlement, retention and independent validation of identity assumptions.

Lab-Safe Case Studies

26.1 Case A: userland ransomware precursor

26.1.1 Scenario

A Windows file server shows rapid file-renaming alerts. EDR records a new process but telemetry becomes incomplete. A live memory image and disk snapshot are acquired before isolation.

26.1.2 Questions

- Which process is performing file operations?
- Was it launched interactively, by a service, task or remote tool?
- Is executable code injected into a legitimate process?
- Are recovery-inhibition commands present?
- Is there evidence of staging or exfiltration before encryption?

26.1.3 Workflow

```
vol.exe -f server.raw windows.info.Info
vol.exe -q -f server.raw -r csv windows.pstree.PsTree > pstree.csv
vol.exe -q -f server.raw -r csv windows.cmdline.CmdLine > cmdline.csv
vol.exe -q -f server.raw -r csv windows.handles.Handles > handles.csv
vol.exe -q -f server.raw -r csv windows.netstat.NetStat > netstat.csv
vol.exe -q -f server.raw -r csv windows.malware.malfind.Malfind > malfind.csv
vol.exe -q -f server.raw -r csv windows.malware.suspicious_threads.SuspiciousThreads > threads.csv
vol.exe -q -f server.raw -r csv windows.svcscan.SvcScan > services.csv
vol.exe -q -f server.raw -r csv windows.registry.scheduled_tasks.ScheduledTasks > tasks.csv
```

26.1.4 Synthetic output

```
Process: payroll-sync.exe PID: 6840 Parent: services.exe
Command: C:\ProgramData\PayrollSync\payroll-sync.exe --job nightly
Handles: hundreds of writable .docx/.xlsx files on D:\Shares
Network: 10.50.20.14:51124 -> 203.0.113.50:443
Malfind: private RX VAD at 0x000001F400000000
Thread: TID 6924 starts at 0x000001F4000012A0
Service: PayrollSync, auto-start, binary in ProgramData
```

The IP and names are documentation-only examples. The convergence of service persistence, high-volume writable file handles, external connection, executable private VAD and thread start inside that VAD supports probable process injection and active impact behavior. Extract the VAD and process, hash them, inspect the service binary, preserve network/EDR logs and isolate the host.

26.1.5 Short finding

Probable malicious encryption process with injected code. Confidence: high. Evidence: memory image, VAD/thread relationship, service configuration, file handles and network endpoint. Immediate actions: isolate, protect backups, revoke associated credentials, preserve peer-host and network telemetry, and stop destructive activity according to the incident commander's decision.

26.2 Case B: suspicious cloud-connected process

26.2.1 Scenario

An EC2 Linux workload produces GuardDuty findings and unusual outbound traffic. The instance remains running. Responders attach an isolation security group, create EBS snapshots, export CloudTrail and acquire guest memory.

26.2.2 Workflow

```
python3 vol.py -f node.lime linux.psaux.PsAux
python3 vol.py -f node.lime linux.proc.Maps
python3 vol.py -f node.lime linux.sockstat.Sockstat
python3 vol.py -f node.lime linux.lsof.Lsof
python3 vol.py -f node.lime linux.malware.malfind.Malfind
python3 vol.py -f node.lime linux.envvars.Envvars
python3 vol.py -f node.lime linux.ebpf.EBPF
```

CloudTrail shows an unusual role session followed by SSM execution and security-group changes. Memory shows a shell process whose environment contains short-lived cloud credentials and whose socket maps to the suspicious endpoint. The incident is not merely an infected process: scope includes the role session, all API calls made with it, SSM history, other instances targeted by the same principal and credentials present in workload configuration.

26.2.3 Conclusion

Contain both the instance and the identity. Rotate credentials, revoke sessions where supported, inspect role trust and policies, preserve snapshots/logs, and rebuild the workload from trusted infrastructure as code.

26.3 Case C: suspected Windows kernel rootkit

26.3.1 Scenario

An EDR alert claims a hidden driver. The memory image contains a module present in ModScan but absent from Modules.

26.3.2 Workflow

```
vol.exe -q -f host.raw -r csv windows.modules.Modules > modules.csv
vol.exe -q -f host.raw -r csv windows.modscan.ModScan > modscan.csv
vol.exe -q -f host.raw -r csv windows.driverscan.DriverScan > drivers.csv
vol.exe -q -f host.raw -r csv windows.malware.drivermodule.DriverModule > drivermodule.csv
vol.exe -q -f host.raw -r csv windows.driverirp.DriverIrp > irp.csv
vol.exe -q -f host.raw -r csv windows.callbacks.Callbacks > callbacks.csv
vol.exe -q -f host.raw -r csv windows.orphan_kernel_threads.Threads > orphan.csv
vol.exe -q -f host.raw -r csv windows.unloadedmodules.UnloadedModules > unloaded.csv
```

The module also appears in the unloaded list, has no active driver object, no callback, no IRP target and no thread. The image was captured live and contains pool artifacts. The correct conclusion is **not confirmed**; likely unloaded residue. A rootkit claim would be unsupported.

26.4 Case D: Azure identity and ransomware hybrid

26.4.1 Scenario

A Windows VM encrypts shared data while Azure logs show a new role assignment and deletion attempts against backups.

26.4.2 Method

1. Preserve Entra sign-ins, audit logs, Activity Log and Defender alerts.
2. Isolate the VM NIC without deallocating it.
3. Snapshot OS/data disks and acquire memory.
4. Analyze process, command line, connections, handles, VADs and persistence.
5. Trace the role assignment and every API call made by the principal.
6. Protect vaults, snapshots and Key Vault; validate immutable controls.
7. Hunt for the same identity, binary, command line and destination across subscriptions and endpoints.

This case demonstrates why endpoint eradication and cloud identity remediation must be performed as one coordinated operation.

Professional Reporting, Remediation and Peer Review

27.1 Report architecture

A publishable report contains:

1. executive summary;
2. scope and authorization;
3. evidence inventory and hashes;
4. acquisition methods and limitations;
5. methodology and tool versions;
6. timeline;
7. findings with evidence and confidence;
8. affected assets and accounts;
9. IOCs and detection recommendations;
10. containment, eradication and recovery actions;
11. residual risk and unanswered questions;
12. appendices with command/output references.

27.2 Finding template

Title	Concise behavior and affected asset
Severity	Operational impact and likelihood, not analyst surprise
Confidence	Confirmed, probable, suspicious, inconclusive or benign
Evidence	Exact image, plugin/output row, log/event ID, snapshot/resource ID
Observation	What the evidence directly shows
Interpretation	Why it matters and competing explanations
Scope	Hosts, accounts, regions, subscriptions, data and time window
MITRE ATT&CK	Only techniques directly supported
Recommendation	Immediate, near-term and strategic actions

Limitations

Missing evidence, acquisition gaps, smearing and provider boundaries

27.3 Executive writing rules

State impact, confidence and required decisions. Avoid dumping plugin names into the executive summary. Example:

High-confidence evidence shows that a service-launched process encrypted files on two servers after receiving code in a private executable memory region. The same account created a new cloud role assignment and attempted to delete recovery snapshots. Immediate priorities are identity containment, backup protection, isolation of the two hosts and enterprise-wide hunting for the binary, command line and role session.

27.4 Remediation principles

- Remove root cause, not only the observed binary.
- Rebuild systems with uncertain kernel or administrative integrity.
- Rotate credentials, keys, certificates, tokens and application secrets based on exposure.
- Revoke persistence in OAuth apps, service principals, roles, federation and automation.
- Validate backups offline before restoration.
- Restore in dependency order and monitor aggressively.
- Preserve evidence until legal, regulatory and business retention requirements are satisfied.

27.5 Peer review

A second analyst should verify:

- image hash and symbol match;
- plugin command and version;
- interpretation of each high-severity finding;
- alternative benign explanations;
- timeline normalization;
- IOC accuracy;
- cloud account/region/subscription scope;
- whether recommendations follow from evidence.

Troubleshooting and Failure Analysis

28.1 Plugin identifier errors

Use the CLI identifier, not the Python import path. Correct:

```
vol.exe -f memory.raw windows.malware.suspicious_threads.SuspiciousThreads
```

Incorrect:

```
vol.exe -f memory.raw volatility3.plugins.windows.malware.\  
suspicious_threads.SuspiciousThreads
```

Always copy the identifier from `vol.exe -h`. Deprecated aliases may exist temporarily and then be removed.

28.2 Missing Windows symbols

- Confirm internet access or pre-stage the required symbol cache.
- Check proxy and TLS interception.
- Confirm the image is not truncated.
- Record the PDB GUID/age shown by `windows.info.Info`.
- Use `-s` only for a deliberate symbol directory; do not mix unknown third-party JSON files.

28.3 Linux symbol mismatch

Symptoms include impossible PIDs, broken lists, nonsensical pointers and widespread plugin failures. Recover the exact banner, obtain matching debug symbols and regenerate the ISF. Do not solve an exact-match problem by renaming an approximate symbol file.

28.4 Empty output

Empty output can mean:

- the condition was not found;
- the relevant pages were not acquired;
- symbols or structures are unsupported;

- the plugin's detection scope does not cover the technique;
- the artifact expired before acquisition;
- an error was redirected to a separate file.

Capture stderr during batch runs and inspect files for **Traceback**, **ERROR**, **WARNING** and **Unsatisfied requirement**. A one-kilobyte file containing only the framework banner can be deleted after verification.

28.5 Corrupt or smeared dump

Indicators include impossible PIDs, noncanonical pointers, inconsistent linked lists and timestamps outside the expected instant. Compare multiple enumerators, use acquisition start/end as a range, and avoid treating one damaged structure as malicious concealment.

28.6 VM snapshot issues

A VM memory file may require conversion, metadata or accompanying state files. Do not rename arbitrary hypervisor files to **.raw**. Preserve all snapshot components and consult the hypervisor's documented format. Validate the output with OS information and several unrelated plugin families.

28.7 CSV and shell redirection

Separate stdout and stderr:

```
vol.exe -q -f memory.raw -r csv windows.pslist.PsList \
  > results\pslist.csv 2> results\pslist.err.txt
```

In PowerShell, store a language-level **foreach** result before piping:

```
$Results = foreach ($File in $Files) {
  if (Test-Path -LiteralPath $File) {
    Get-FileHash -LiteralPath $File -Algorithm SHA256
  }
}
$Results | Format-Table
```

28.8 Operational permission failures

Windows acquisition may require administrative rights and driver loading. Linux acquisition can be blocked by lockdown, Secure Boot policy, missing kernel headers or module-signing requirements. Cloud acquisition requires explicit IAM/RBAC permissions and KMS/Key Vault access. Treat workarounds as change-controlled security decisions, not convenience steps.

Professional Workflow Checklists

29.1 Pre-incident readiness

- ☐ Written authorization and emergency contacts
- ☐ Tested Windows/Linux acquisition packages stored offline
- ☐ Volatility version and Python wheels archived with hashes
- ☐ Windows symbol cache and Linux ISF process tested
- ☐ Forensic storage sized for RAM and snapshots
- ☐ AWS forensic account/roles/isolation groups prepared
- ☐ Azure forensic subscription/roles/isolation NSGs prepared
- ☐ Centralized identity, cloud and network logging enabled
- ☐ Immutable backup and evidence-retention controls tested
- ☐ Ransomware tabletop and restore exercise completed

29.2 Acquisition checklist

- ☐ Record case ID, host, operator, UTC time and authorization
- ☐ Photograph/document system state where appropriate
- ☐ Decide volatile-first versus immediate containment
- ☐ Record acquisition command and tool hash/version
- ☐ Capture start/end and errors
- ☐ Hash source output and transferred copy
- ☐ Preserve cloud snapshot/resource/event identifiers
- ☐ Store original read-only; analyze working copy
- ☐ Document expected footprint and omissions

29.3 Triage checklist

- ☐ OS/kernel/symbol validation
- ☐ Process lists, scan and tree

- ☐ Command lines, users, sessions and privileges
- ☐ Active and residual network endpoints
- ☐ Modules/libraries and executable private memory
- ☐ Injection, hollowing, ghosting and thread checks
- ☐ Services/tasks/registry or Linux persistence
- ☐ Kernel modules/drivers and callbacks/hooks
- ☐ Ransomware impact and recovery-inhibition indicators
- ☐ Cloud identity/control-plane activity where applicable

29.4 Deep-analysis checklist

- ☐ Hypothesis and candidate ranking documented
- ☐ VAD/map boundaries and protection validated
- ☐ Thread start mapped to exact allocation/module
- ☐ Candidate extracted and hashed
- ☐ Static/reverse engineering performed in isolated lab
- ☐ Signer and disk provenance checked
- ☐ Network and identity chain reconstructed
- ☐ Persistence and lateral movement scoped
- ☐ Baseline comparison completed
- ☐ Alternative explanations tested

29.5 Reporting checklist

- ☐ Evidence inventory and hashes included
- ☐ Tool versions and command references included
- ☐ Facts separated from inference and intelligence
- ☐ Confidence stated for every material finding
- ☐ Scope and limitations explicit
- ☐ IOCs validated and normalized
- ☐ MITRE mapping evidence-based
- ☐ Immediate and long-term recommendations separated
- ☐ Peer review completed
- ☐ Sensitive credential and personal data minimized

Command Reference

30.1 Environment and output

Command	Purpose
<code>vol.exe--version</code>	Record framework version
<code>vol.exe-h</code>	List installed plugin identifiers and global options
<code>vol.exePLUGIN-h</code>	Verify plugin-specific options
<code>vol.exe-q-fIMAGE-rcsvPLUGIN</code>	Quiet CSV output for repeatable analysis
<code>vol.exe-sSYMBOLS-pPLUGINS-fIM AGEPLUGIN</code>	Explicit symbol and plugin directories
<code>frameworkinfo.FrameworkInfo</code>	Framework component/version information
<code>isfinfo.IsfInfo</code>	Inspect symbol-table metadata
<code>banners.Banners</code>	Find OS/kernel banners where supported

30.2 Windows triage

Plugin	Use
<code>windows.info.Info</code>	Kernel, build, DTB, architecture and symbols
<code>windows.pslist.PsList</code>	Active process list
<code>windows.psscan.PsScan</code>	Pool scan for process objects
<code>windows.pstree.PsTree</code>	Parent-child process tree
<code>windows.malware.psxview.PsXView</code>	Cross-view process comparison
<code>windows.cmdline.CmdLine</code>	Process command lines
<code>windows.getsids.GetSIDs</code>	Security identifiers
<code>windows.privileges.Privs</code>	Token privileges
<code>windows.sessions.Sessions</code>	Session association
<code>windows.joblinks.JobLinks</code>	Job object relationships
<code>windows.handles.Handles</code>	Handles and named kernel objects
<code>windows.threads.Threads</code>	Process threads
<code>windows.thrdscan.ThrdScan</code>	Pool scan for thread objects

30.3 Windows modules and injection

Plugin	Use
<code>windows.dlllist.DllList</code>	Loader-listed modules and optional dumping
<code>windows.malware.ldrmodules.LdrModules</code>	Compare VAD images with loader lists
<code>windows.vadinfo.VadInfo</code>	VAD boundaries, protection and file mapping
<code>windows.vadwalk.VadWalk</code>	Walk VAD tree
<code>windows.memmap.Memmap</code>	Process virtual-to-physical map and dump
<code>windows.malware.malfind.Malfind</code>	Potential injected/private executable regions
<code>windows.malware.suspicious_threads.SuspiciousThreads</code>	Threads with suspicious VAD relationships
<code>windows.suspended_threads.SuspendedThreads</code>	Suspended thread inventory
<code>windows.malware.hollowprocesses.HollowProcesses</code>	Hollowing indicators
<code>windows.malware.processghosting.ProcessGhosting</code>	Process ghosting indicators
<code>windows.malware.pebmasquerade.PebMasquerade</code>	PEB image/command-line mismatch
<code>windows.etwpatch.EtwPatch</code>	ETW patch indicators
<code>windows.malware.direct_system_calls.DirectSystemCalls</code>	Direct syscall stubs outside expected modules
<code>windows.malware.indirect_system_calls.IndirectSystemCalls</code>	Indirect syscall indicators
<code>windows.malware.unhooked_system_calls.UnhookedSystemCalls</code>	Alternate syscall implementations
<code>windows.vadyarascan.VadYaraScan</code>	YARA scan process VADs

30.4 Windows network and persistence

Plugin	Use
<code>windows.netstat.NetStat</code>	Active socket/endpoints from network structures
<code>windows.netscan.NetScan</code>	Pool scan for network artifacts
<code>windows.svcscan.SvcScan</code>	Service object scan
<code>windows.svclist.SvcList</code>	Service list
<code>windows.malware.svcdiff.SvcDiff</code>	Service enumeration differences
<code>windows.registry.scheduled_tasks.ScheduledTasks</code>	Scheduled task artifacts
<code>windows.registry.hivelist.HiveList</code>	Registry hive inventory
<code>windows.registry.hivescan.HiveScan</code>	Registry hive scan
<code>windows.registry.printkey.PrintKey</code>	Targeted registry key/value view

<code>windows.amcache.Amcache</code>	Amcache execution/file metadata
<code>windows.shimcachemem.ShimcacheMem</code>	Shimcache data in memory
<code>windows.registry.userassist.UserAssist</code>	UserAssist artifacts

30.5 Windows kernel

Plugin	Use
<code>windows.modules.Modules</code>	Active loaded kernel modules
<code>windows.modscan.ModScan</code>	Pool-scan module structures
<code>windows.driverscan.DriverScan</code>	Driver objects
<code>windows.malware.drivermodule.DriverModule</code>	Driver-object to module mapping
<code>windows.driverirp.DriverIrp</code>	IRP dispatch routines
<code>windows.callbacks.Callbacks</code>	Kernel notification callbacks
<code>windows.ssdt.SSDT</code>	System service dispatch table
<code>windows.orphan_kernel_threads.Threads</code>	Kernel threads outside known modules
<code>windows.timers.Timers</code>	Kernel timers and routines
<code>windows.unloadedmodules.UnloadedModules</code>	Recently unloaded modules
<code>windows.devicetree.DeviceTree</code>	Driver/device relationships
<code>windows.symlinkscan.SymlinkScan</code>	Symbolic link objects
<code>windows.bigpools.BigPools</code>	Big pool allocations
<code>windows.poolscanner.PoolScanner</code>	Advanced targeted pool scanning
<code>windows.mbrscan.MBRScan</code>	MBR signatures where relevant

30.6 Linux reference

Plugin	Use
<code>linux.pslist.PsList</code> , <code>linux.psscan.PsScan</code> , <code>linux.pstree.PsTree</code>	Process enumeration
<code>linux.psaux.PsAux</code> , <code>linux.envvars.Envvars</code>	Arguments and environment
<code>linux.proc.Maps</code> , <code>linux.library_list.LibraryList</code>	Process mappings and libraries
<code>linux.malware.malfind.Malfind</code>	Potential injected mappings
<code>linux.lsof.Lsof</code> , <code>linux.mountinfo.MountInfo</code>	Open files and mounts
<code>linux.sockstat.Sockstat</code> , <code>linux.sockscan.Sockscan</code>	Socket artifacts
<code>linux.ip.Addr</code> , <code>linux.ip.Link</code>	Interface/address information
<code>linux.lsmmod.Lsmmod</code> , <code>linux.hidden_modules.HiddenModules</code> , <code>linux.modxview.Modxview</code>	Module cross-views
<code>linux.check_syscall.Check_syscall</code> , <code>linux.check_idt.Check_idt</code>	Hook detection
<code>linux.check_afinfo.Check_afinfo</code> , <code>linux.netfilter.Netfilter</code>	Network hook/filter analysis

<code>linux.ebpf.EBPF</code> , <code>linux.tracing.ftrace.CheckFtrace</code>	eBPF and tracing state
<code>linux.pagecache.Files</code> , <code>linux.pagecache.RecoverFs</code>	Page-cache inventory/recovery

30.7 macOS reference

Plugin	Use
<code>mac.pslist.PsList</code> , <code>mac.pstree.PsTree</code> , <code>mac.psaux.Psaux</code>	Process analysis
<code>mac.proc_maps.Maps</code> , <code>mac.malfind.Malfind</code>	Memory maps and injection leads
<code>mac.lsof.Lsof</code> , <code>mac.list_files.List_Files</code>	File artifacts
<code>mac.netstat.Netstat</code> , <code>mac.ifconfig.Ifconfig</code>	Network artifacts
<code>mac.lsmod.Lsmod</code>	Kernel modules
<code>mac.check_syscall.Check_syscall</code> , <code>mac.check_trap_table.Check_trap_table</code>	Kernel table checks
<code>mac.kauth_listeners.Kauth_listeners</code> , <code>mac.kauth_scopes.Kauth_scopes</code>	Kauth state
<code>mac.socket_filters.Socket_filters</code> , <code>mac.trustedbsd.Trustedbsd</code>	Security/network hooks

Final Recommendations for 2026

31.1 Build competence in layers

A strong memory-forensics analyst needs operating-system internals, incident response, network analysis, malware reverse engineering, cloud identity and evidence handling. Tool familiarity without structure knowledge produces false positives; structure knowledge without incident context produces technically correct but operationally weak reports.

31.2 Maintain a versioned laboratory

Keep:

- pinned Volatility releases and Python dependencies;
- archived help output for each release;
- known-good memory images by OS build;
- clean baselines for browsers, runtimes, EDR and common drivers;
- tested Windows symbol cache and Linux ISFs;
- offline acquisition packages and hashes;
- AWS/Azure test accounts for playbook validation;
- ransomware simulations using harmless files and synthetic indicators.

31.3 Practice evidence-driven restraint

The professional skill is not finding something unusual; it is proving what the unusual artifact means. Refuse unsupported rootkit, attribution or clean-system claims. State limitations directly and request the missing evidence plane.

31.4 Recommended core references

Study the official Volatility documentation and source, NIST SP 800-61 Revision 3, NIST SP 800-86, the 2026 NIST ransomware profile, CISA StopRansomware guidance, Windows Internals, Linux kernel documentation, AWS Security Incident Response guidance, Azure incident-response guidance, MITRE ATT&CK, YARA and Sigma.[\[2, 3, 5, 4, 16, 12, 24, 35, 36, 37\]](#)

31.5 Closing principle

Volatile memory is a high-value but perishable witness. Acquire it lawfully, preserve it rigorously, analyze it with multiple independent views, correlate it across endpoint, network, identity and cloud evidence, and report only what the evidence supports.

Evidence Manifest Template

Case ID
Evidence ID
Description
Source system/resource
Acquisition tool/version
Acquisition com-
mand/API
Operator
Start/end UTC
Source time zone
Original filename/re-
source ID
Size
SHA-256
Storage location
Access restrictions
Known acquisition effects
Notes

Chain-of-Custody Template

UTC time	Evidence ID	Released by/to	Action/location	Signature/reference
----------	-------------	----------------	-----------------	---------------------

Finding Confidence Matrix

Confidence	Evidence expectation	Example language
Confirmed	Direct artifact plus independent validation; alternatives excluded	The process executed code from the extracted private VAD, as shown by thread start and matching instructions.
Probable	Multiple converging indicators; a material uncertainty remains	Evidence strongly supports process injection, although the allocator could not be identified.
Suspicious	Anomaly with plausible benign alternatives	The anonymous executable VAD warrants extraction and baseline comparison.
Inconclusive	Evidence is insufficient or contradictory	The module-list difference cannot distinguish unload residue from concealment.
Benign/expected	Validated product/OS mechanism or clean baseline	The repeated browser syscall table matches the same version on a clean baseline.

Glossary

API	Application programming interface
C2	Command and control
DKOM	Direct Kernel Object Manipulation
DTB	Directory Table Base used for address translation
EDR	Endpoint detection and response
ENI	Elastic Network Interface in AWS
EBS	Elastic Block Store
IAT	Import Address Table
IDT	Interrupt Descriptor Table
IOMMU	Input-output memory management unit
IRP	I/O Request Packet
ISF	Volatility Intermediate Symbol File
JIT	Just-in-time code generation
KQL	Kusto Query Language
NSG	Network Security Group in Azure
PEB	Process Environment Block
SSDT	System Service Descriptor Table
VAD	Virtual Address Descriptor
VBS	Virtualization-Based Security

Version and Validation Record

Manual publication date	01.08.2026
Volatility baseline	2.28.0 stable
Command validation	Official 2.28.0 documentation, plugin source and installed CLI help
Windows scope	Supported Windows memory images with matching symbols; Windows 11 included subject to build/acquisition quality
Linux scope	Exact kernel banner and matching ISF required
macOS scope	Research/legacy scope; official support not actively maintained
Cloud scope	AWS and Azure customer-controlled evidence planes; provider internals excluded
Safety scope	Authorized defensive analysis, incident response and education only

Bibliography

- [1] Volatility Foundation. *Volatility 3 Releases*. URL: <https://github.com/volatilityfoundation/volatility3/releases> (visited on 08/01/2026).
- [2] Volatility Foundation. *Volatility 3 2.28.0 Documentation*. URL: <https://volatility3.readthedocs.io/en/stable/> (visited on 08/01/2026).
- [3] Alexander Nelson, Sushil Rekhi, Murugiah Souppaya, and Karen Scarfone. *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*. Tech. rep. NIST SP 800-61 Rev. 3. National Institute of Standards and Technology, 2025. URL: <https://csrc.nist.gov/pubs/sp/800/61/r3/final> (visited on 08/01/2026).
- [4] National Institute of Standards and Technology. *Ransomware Risk Management: A Cybersecurity Framework 2.0 Community Profile*. Tech. rep. NIST IR 8374 Revision 1. National Institute of Standards and Technology, 2026. URL: <https://csrc.nist.gov/pubs/ir/8374/r1/final> (visited on 08/01/2026).
- [5] Karen Kent, Suzanne Chevalier, Tim Grance, and Hung Dang. *Guide to Integrating Forensic Techniques into Incident Response*. Tech. rep. NIST SP 800-86. National Institute of Standards and Technology, 2006. URL: <https://csrc.nist.gov/pubs/sp/800/86/final> (visited on 08/01/2026).
- [6] Volatility Foundation. *Volatility 3 macOS Tutorial*. URL: <https://volatility3.readthedocs.io/en/stable/getting-started-mac-tutorial.html> (visited on 08/01/2026).
- [7] Velocidex. *WinPmem: Physical Memory Acquisition Tool*. URL: <https://github.com/Velocidex/WinPmem> (visited on 08/01/2026).
- [8] Microsoft. *Overview of Memory Dump File Options for Windows*. URL: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/performance/memory-dump-file-options> (visited on 08/01/2026).
- [9] Microsoft. *Force a System Crash from the Keyboard*. URL: <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/forcing-a-system-crash-from-the-keyboard> (visited on 08/01/2026).
- [10] Microsoft. *AVML: Acquire Volatile Memory for Linux*. URL: <https://github.com/microsoft/avml> (visited on 08/01/2026).
- [11] Linux Kernel Documentation Project. *Kdump: The kexec-based Crash Dumping Solution*. URL: <https://docs.kernel.org/admin-guide/kdump/kdump.html> (visited on 08/01/2026).
- [12] Amazon Web Services. *AWS Security Incident Response User Guide*. URL: <https://docs.aws.amazon.com/security-ir/latest/userguide/what-is-security-ir.html> (visited on 08/01/2026).
- [13] Volatility Foundation. *dwarf2json*. URL: <https://github.com/volatilityfoundation/dwarf2json> (visited on 08/01/2026).
- [14] Volatility Foundation. *Volatility 3 Linux Tutorial*. URL: <https://volatility3.readthedocs.io/en/stable/getting-started-linux-tutorial.html> (visited on 08/01/2026).

- [15] Federal Bureau of Investigation. *2025 IC3 Annual Report*. Internet Crime Complaint Center, 2026. URL: https://www.ic3.gov/AnnualReport/Reports/2025_IC3Report.pdf (visited on 08/01/2026).
- [16] Cybersecurity and Infrastructure Security Agency. *StopRansomware Guide*. URL: <https://www.cisa.gov/stopransomware/ransomware-guide> (visited on 08/01/2026).
- [17] Federal Bureau of Investigation. *Ransomware: What It Is and What To Do About It*. URL: <https://www.fbi.gov/how-we-can-help-you/scams-and-safety/common-frauds-and-scams/ransomware> (visited on 08/01/2026).
- [18] United States Department of Justice. *Department of Justice Seizes 2.3 Million Dollars in Cryptocurrency Paid to the Ransomware Extortionists DarkSide*. 2021. URL: <https://www.justice.gov/opa/pr/departement-justice-seizes-23-million-cryptocurrency-paid-ransomware-extortionists-darkside> (visited on 08/01/2026).
- [19] UnitedHealth Group Incorporated. *Current Report on Form 8-K Concerning Change Healthcare Cybersecurity Incident*. 2024. URL: <https://www.sec.gov/Archives/edgar/data/731766/000073176624000045/unh-20240221.htm> (visited on 08/01/2026).
- [20] Cybersecurity and Infrastructure Security Agency. *ESXiArgs Ransomware Virtual Machine Recovery Guidance*. 2023. URL: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-039a> (visited on 08/01/2026).
- [21] Amazon Web Services. *Incident Response and Forensics - Amazon EKS Best Practices Guide*. URL: <https://docs.aws.amazon.com/eks/latest/best-practices/incident-response-and-forensics.html> (visited on 08/01/2026).
- [22] 504ensics Labs. *LiME: Linux Memory Extractor*. URL: <https://github.com/504ensicsLabs/LiME> (visited on 08/01/2026).
- [23] Amazon Web Services. *Automated Forensics Orchestrator for Amazon EC2 and EKS*. URL: <https://docs.aws.amazon.com/solutions/latest/automated-forensics-orchestrator-for-amazon-ec2/solution-overview.html> (visited on 08/01/2026).
- [24] Microsoft. *Incident Response Overview for Azure*. URL: <https://learn.microsoft.com/en-us/azure/security/fundamentals/incident-response-overview> (visited on 08/01/2026).
- [25] Microsoft. *Microsoft Cloud Security Benchmark v2 - Incident Response*. URL: <https://learn.microsoft.com/en-us/security/benchmark/azure/mcsb-v2-incident-response> (visited on 08/01/2026).
- [26] Microsoft. *Computer Forensics Chain of Custody in Azure*. URL: <https://learn.microsoft.com/en-us/azure/architecture/example-scenario/forensics/> (visited on 08/01/2026).
- [27] Microsoft. *Activity Log in Azure Monitor*. URL: <https://learn.microsoft.com/en-us/azure/azure-monitor/platform/activity-log> (visited on 08/01/2026).
- [28] Microsoft. *What Is Microsoft Sentinel?* URL: <https://learn.microsoft.com/en-us/azure/sentinel/overview> (visited on 08/01/2026).
- [29] Microsoft. *Hunt for Threats with Microsoft Sentinel*. URL: <https://learn.microsoft.com/en-us/azure/sentinel/hunting> (visited on 08/01/2026).
- [30] MGM Resorts International. *Current Report on Form 8-K: Cybersecurity Incident Update*. 2023. URL: <https://www.sec.gov/Archives/edgar/data/789570/000119312523251667/d461062d8k.htm> (visited on 08/01/2026).
- [31] Cybersecurity and Infrastructure Security Agency. *Scattered Spider*. 2023. URL: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-320a> (visited on 08/01/2026).
- [32] United States Department of Justice. *Former Hacker Sentenced for Stealing Computer Power and Personal Information of More Than 100 Million People*. 2022. URL: <https://www.justice.gov/usao-wdwa/pr/former-hacker-sentenced-stealing-computer-power-mine-cryptocurrency-and-stealing> (visited on 08/01/2026).

- [33] Microsoft Threat Intelligence. *Midnight Blizzard: Guidance for Responders on Nation-State Attack*. 2024. URL: <https://www.microsoft.com/en-us/security/blog/2024/01/25/midnight-blizzard-guidance-for-responders-on-nation-state-attack/> (visited on 08/01/2026).
- [34] Cyber Safety Review Board. *Review of the Summer 2023 Microsoft Exchange Online Intrusion*. Cybersecurity and Infrastructure Security Agency, 2024. URL: <https://www.cisa.gov/resources-tools/resources/review-summer-2023-microsoft-exchange-online-intrusion> (visited on 08/01/2026).
- [35] MITRE. *MITRE ATT&CK Enterprise Techniques*. URL: <https://attack.mitre.org/techniques/enterprise/> (visited on 08/01/2026).
- [36] VirusTotal. *YARA Documentation*. URL: <https://yara.readthedocs.io/> (visited on 08/01/2026).
- [37] SigmaHQ. *Sigma Rule Specification*. URL: <https://sigmahq.io/docs/basics/rules.html> (visited on 08/01/2026).